



École Polytechnique Fédérale de Lausanne

Adaptive Calibration Algorithms
for Tunable Analog Peripherals

by Khalil M'hirsi

Master Thesis

Approved by the Examining Committee:

Prof. Mathieu Salzmann
Thesis Advisor

Richard Lengagne
External Expert

Christophe Constantin
Thesis Supervisor

EPFL CVLAB
BC 309 (Bâtiment BC)
Station 14
CH-1015 Lausanne

7 August 2026

*“Pour ceux qui
viendront après.”
— Clair Obscur, Expedition 33*

Acknowledgments

My thanks go first to Christophe Constantin, my supervisor at Logitech, who trusted this project enough to let it take its own direction and never let a weak argument pass without pushing back on it. I also thank Professor Mathieu Salzmann for holding the work to a standard that kept it honest and on track.

Kilian and Frederic helped on the hardware side more times than I can count, and Jozef and Lilou were always up for a conversation about machine learning or signal processing that ended up mattering more than either of us expected. I am also grateful for Yael's eye for UX and interface design, who helped me shape up the minigames and recommendation views. Thanks as well to the wider team at Logitech for the discussions, the advice, and the work of organizing data-collection sessions and user tests that this thesis depended on.

Sophia, my partner, carried more of this than she should have had to during a hard stretch, and I would not have finished it without her. Hugo, Christelle, and Maxence kept me sane and still going when labeling the data. And to everyone else who stayed close through a difficult time: thank you for helping me see this through.

Lausanne, 7 August 2026

Khalil M'hirsi

Abstract

Analog peripherals with continuous depth-sensing hardware offer sub-millimetre actuation precision, yet their firmware often relies on static thresholds that ignore per-user biomechanical variance and degradation under neuromotor fatigue. This thesis addresses that gap by designing an end-to-end click-sensitivity recommendation pipeline for Haptic Inductive Trigger System (HITS) devices, evaluated on a set of 26 participants.

The system makes three contributions. (1) A supervised labeling workflow that bootstraps frame-level annotations from algorithmic criteria and iteratively improves them through model-assisted refinement, substantially reducing the manual annotation burden while preserving the boundary precision needed for downstream training. (2) A four-class temporal intent-detection model supporting three architectures: a Random Forest and a Gradient Boosted Trees variant operating on hand-crafted sliding-window features, and a dilated residual FCN operating directly on raw telemetry with a large learned receptive field. The Random Forest achieves the best deployment performance (F1 0.970 on the left channel), and a probe ablation shows that raw local context around each frame is a far stronger predictor than explicit kinematic derivative features. (3) A firmware-faithful threshold optimizer that exhaustively evaluates the discrete (AP, RT) grid via a Rapid Trigger FSM simulator, replacing manual threshold tuning with a data-driven recommendation. Across the annotated corpus, the optimizer consistently reduces make latency and increases the true-positive detection rate relative to the factory default, with the improvement holding across every session evaluated.

Contents

Acknowledgments	2
Abstract (English/Français)	3
1 Introduction	7
1.1 The Evolution of Continuous Sensing Hardware	7
1.2 The Limitations of Static Actuation Thresholds	7
1.3 Neuromotor Fatigue and Performance Decay	7
1.4 Toward Adaptive Intent Detection	7
1.5 Thesis and Contributions	7
2 Background	8
2.1 Continuous Depth-Sensing Mechanisms	8
2.1.1 Hall-Effect and Magnetic Sensing	8
2.1.2 Haptic Inductive Trigger System (HITS)	8
2.2 Control Parameters: AP and RT	8
2.2.1 Actuation Point (AP)	8
2.2.2 Rapid Trigger (RT)	9
2.3 Biomechanical Variance and Neuromotor Fatigue	9
2.3.1 Resting Pressure and Tremors	9
2.3.2 Neuromotor Fatigue Signatures	9
2.4 Constraints of Offline Calibration (Scope Context)	9
2.5 Signal Processing and Optimisation Foundations	9
2.5.1 Savitzky-Golay Smoothing	9
2.5.2 Hermite Cubic Interpolation: Akima and Makima	10
2.5.3 Covariance Matrix Adaptation Evolution Strategy	10
2.5.4 Bayesian Search and Threshold Sweeping	10
2.5.5 Ensemble Methods: Random Forests and Gradient Boosting	10
2.5.6 Dilated Fully Convolutional Networks	11
3 Related Work	11
3.1 Adaptive Threshold Methods for HID Devices	11
3.2 Hyperparameter and Threshold Optimization	11
3.3 Temporal Sequence Labeling in Biosignal Processing	12
3.4 Weak Supervision and Algorithmic Bootstrapping	12
3.5 Dilated Convolutions for Sequence Modeling	13
4 Design	13
4.1 System Architecture	13
4.2 Data Labeling Methodology	13
4.2.1 Algorithmic Bootstrapping	14
4.2.2 Iterative Human-in-the-Loop Annotation	14
4.3 Data Preparation Pipeline	15
4.4 Click-Intent Sequence Modeling	15
4.4.1 Random Forest	15
4.4.2 Gradient Boosted Trees	16

4.4.3	Fully Convolutional Network	16
4.5	From Classification to Segmentation	16
4.6	Parameter Optimization	17
4.6.1	Firmware State Machine Simulation	17
4.6.2	Search Space and Objective	17
4.6.3	Pareto Frontier and Output	18
5	Implementation	18
5.1	Data Acquisition Campaign	18
5.1.1	Controlled Task Dataset (BIC Event)	18
5.1.2	Free-Play Dataset (PolyLAN Event)	18
5.1.3	Annotated Dataset Summary	19
5.2	Frontend and Telemetry Stack	19
5.2.1	Analog Decoder and Translator	19
5.2.2	Labeling Interface	19
5.3	Machine Learning Backend	20
5.3.1	Loading and Frame Extraction	20
5.3.2	Preprocessing and Signal Augmentation	20
5.3.3	Splitting, Normalization, and Sample Weighting	21
5.3.4	Feature Extraction and Model Ingestion	21
5.3.5	Gradient Boosted Trees	22
5.4	From Classification to Segmentation	22
5.4.1	Candidate Run Detection	22
5.4.2	Onset Frame Election	23
5.4.3	Make-Break Pairing	23
5.4.4	Hyperparameter Selection via Bayesian Sweep	23
5.5	Optimization Pipeline Integration	23
6	Evaluation	25
6.1	Experimental Setup	25
6.1.1	Dataset and Splits	25
6.1.2	Baselines	25
6.1.3	Metrics	26
6.2	Classification and Segmentation Results	26
6.2.1	Deployment Performance on the Test Set	26
6.2.2	Cross-Participant Generalization (5-Fold Grouped CV)	27
6.2.3	Training Dynamics	27
6.3	Segmentation Configuration	29
6.3.1	Optimal Thresholds and Strategies	29
6.3.2	Pairing Strategy Comparison	29
6.3.3	Channel and Probe Ablation (Random Forest)	30
6.4	Optimizer Evaluation	31
6.4.1	Comparison with Static Default	31
6.5	End-to-End User Study	32
6.5.1	Protocol	32
6.5.2	Results	33
6.5.3	Recommendation Behavior	34

6.5.4	Study Limitations	34
6.6	Discussion	35
7	Conclusion	37
7.1	Summary	37
7.2	Limitations	38
7.3	Future Work	39

1. Introduction

Human-Computer Interaction (HCI) in high-performance computing and competitive esports places stringent demands on input peripherals. In environments where users execute several hundred Actions Per Minute (APM), actuation latency and threshold stability can affect both task performance and musculoskeletal load [esports_tendinopathy, wrist_fatigue_genre_2025].

1.1 The Evolution of Continuous Sensing Hardware

Peripheral hardware has shifted from binary mechanical switches toward continuous sensing interfaces. In HCI, threshold selection and activation logic have long been recognized as key determinants of usability and speed [touchpad_comparison, impact2018]. Recent Hall-effect and inductive devices expose depth-dependent actuation and reset behavior to firmware, including Rapid Trigger mechanisms for dynamic re-actuation [wooting_rapid_trigger, pcgamer_hall_effect]. Commercial systems such as HITS further combine inductive sensing with configurable haptic feedback [logitech_superstrike_2026, logitech_hits_patent, fox_superstrike_2026, techradar_superstrike_2026]. Collectively, these developments increase control resolution but also increase dependence on well-calibrated software thresholds [dynamic_keyboard, attackshark_taphold].

1.2 The Limitations of Static Actuation Thresholds

Despite these hardware leaps, the software interpreting this data remains limited. Current analog peripherals rely on static firmware thresholds that are agnostic to human biomechanical variance. Players exhibit unique control signatures, including micro-hesitations and varying velocity curves, which often clash with rigid hardware logic [impact2018]. When static thresholds are set too aggressively, they fail to filter out physiological noise; when set too conservatively, they introduce unnecessary mechanical pre-travel. This friction

results in sub-optimal latency or a high frequency of unintended actuations during high-stress scenarios [touchpad_comparison, dynamic_keyboard].

1.3 Neuromotor Fatigue and Performance Decay

These hardware trends also intersect with user health. Higher APM in esports has been associated with repetitive strain injuries and tendinopathies [esports_tendinopathy, diffrancisco2019managing, 1hp_esports_ergonomics]. Biomechanical studies report distinct fatigue patterns for keyboard and mouse interaction [fatigue_flexor], and repetitive clicking can degrade aiming accuracy while increasing forearm muscle load [clicking_fatigue_ergonomics_2024]. High-APM genres are linked to elevated kinematic demand on wrist extensors [wrist_fatigue_genre_2025], with measurable neuromotor adaptation during fatiguing tasks [heimhofer2024finger]. These findings motivate adaptive calibration that maintains responsiveness while reducing unintended actuation under fatigue.

1.4 Toward Adaptive Intent Detection

One longer-term direction is intent inference before full mechanical actuation. Prior work on surface electromyography (sEMG) and embedded TinyML suggests this is technically plausible [reactemg, lda_emg, tinymml_mcu_inference, singh2023edge]. However, this thesis focuses on a narrower and directly evaluable scope: offline calibration from depth telemetry produced by the peripheral. On-device inference is treated as future work (Section 7.3). Involving varied gaming scenarios, fatigue detection, and UX integration are likewise positioned as future work.

1.5 Thesis and Contributions

The central hypothesis of this thesis is that per-user variation in click telemetry is structured enough to support automatic labeling, statistical modeling, and parameter optimization beyond static default settings. The contributions are:

- A hybrid labeling pipeline for analog HID click intent that derives initial frame-level MAKE_ONSET,

CLICK_HOLD, and BREAK_ONSET annotations from algorithmic criteria (geometry, speed, and acceleration) and then improves boundary quality through model-assisted iteration.

- A four-class temporal sequence classifier with two model paths: a shallow path (Random Forest, Gradient Boosted Trees) operating on hand-crafted sliding-window features for sub-second per-session training, and a deep path (dilated residual FCN) operating on raw telemetry with a learned receptive field. This classifier is then used to segment click telemetry into labeled sequences for downstream analysis.
- A firmware-faithful threshold optimizer that exhaustively evaluates the discrete (AP, RT) grid via a Rapid Trigger FSM simulator, returning the minimum-loss configuration and a Pareto frontier over detection rate and timing accuracy, replacing manual threshold tuning with a data-driven trade-off curve.

While not a primary algorithmic contribution, the work also includes a small HCI-facing component: purpose-built minigames that recreate common competitive-gaming input scenarios, providing a structured environment for users to generate calibration telemetry and receive a hardware recommendation. The recommendation outputs are structured for interpretability (trade-off views and comparative candidate points), so users can inspect why a suggested AP/RT point is proposed.

2. Background

To understand the design and implementation of an adaptive actuation system, it is necessary to establish the technical foundations of continuous sensing hardware, the mathematical logic of modern trigger parameters, and the biomechanical constraints of the human hand.

2.1 Continuous Depth-Sensing Mechanisms

Traditional mechanical switches operate on a binary principle: a circuit is either open or closed based on a physical contact point. In contrast, continuous depth-sensing switches measure the absolute position of the stem throughout its entire travel distance.

2.1.1 Hall-Effect and Magnetic Sensing. Hall-effect sensors utilize a permanent magnet attached to the switch stem and a sensor on the Printed Circuit Board (PCB). As the magnet approaches the sensor, the Hall voltage varies proportionally to the magnetic field strength [[wooting_rapid_trigger](#), [pcgamer_hall_effect](#)]. This allows the firmware to map voltage levels to precise displacement values in millimeters. Compared to classic contact switches, this technology reduces dependence on debounce filtering and enables lower-latency signal processing.

2.1.2 Haptic Inductive Trigger System (HITS). Refined in the 2026 Logitech SUPERSTRIKE series, the Haptic Inductive Trigger System (HITS) replaces magnets with an inductive coil architecture [[logitech_hits_patent](#), [techradar_superstrike_2026](#)]. By measuring changes in electromagnetic induction caused by the proximity of a metallic puck within the trigger, HITS provides higher resolution and lower susceptibility to external magnetic interference compared to Hall-effect sensors. Furthermore, HITS integrates localized haptic motors to simulate tactile "bumps" or "clicks" at software-defined depths, decoupling the physical feel of the switch from its electrical actuation point [[fox_superstrike_2026](#)].

2.2 Control Parameters: AP and RT

The transition to continuous sensing introduced two primary variables that define the "feel" and speed of an input device.

2.2.1 Actuation Point (AP). The Actuation Point (AP) is the specific depth (d) at which a button-press event is registered. In traditional switches, this is fixed (e.g., 2.0 mm). In analog systems, AP is a variable $d_{act} \in [0.065, 0.65]$ mm [[digitec_logitech_clickery_2024](#)]. For gaming mice, actuation is also commonly characterized by force and total travel: typical actuation force is 55–75 gf, and total click travel is approximately 1.0–1.6 mm. While these parameters are characteristically expressed as spatial displacement for keyboards, in the context of mice, a specific actuation depth inherently corresponds to a specific physical force applied onto the keyplates. Lowering

d_{act} reduces mechanical pre-travel and generally lowers the required actuation force, allowing for faster response times, but increases the risk of accidental actuations caused by resting finger weight.

2.2.2 Rapid Trigger (RT). Rapid Trigger (RT) eliminates the fixed reset point found in mechanical switches. In a standard switch, the button must travel back up past a specific reset point before it can be pressed again. RT logic instead monitors the direction of travel. As soon as the stem moves upward by a specific threshold (Δ_{up}), the input resets. Conversely, as soon as it moves back down by Δ_{down} , it reactuates [**wooting_rapid_trigger**]. RT mechanisms differ primarily in their reference boundaries: dynamic RT algorithms track the deepest and highest positions continuously along the entire travel path, only terminating this dynamic tracking when the displacement reaches absolute zero (a complete release). By contrast, other RT implementations restrict this dynamic tracking exclusively to the domain below the Actuation Point (AP), instantly terminating the rapid trigger state if the stem ascends past the initial AP threshold. Both methods allow for rapid repeated clicking, which is critical in high-APM genres.

2.3 Biomechanical Variance and Neuromotor Fatigue

The efficiency of AP and RT is limited by both hardware imperfections and the physiological state of the user. Before examining human motor control, it is important to acknowledge that the raw sensor data is inherently noisy due to analog-to-digital converter (ADC) polling jitter, quantization noise, and non-linearities at the physical extremes of switch travel (top-out and bottom-out crush). However, even assuming a perfectly linear sensor, human motor control is subject to "signal-dependent noise," where the variance of movement increases with the force or speed of the action [**harris_wolpert_1998**].

2.3.1 Resting Pressure and Tremors. Even at rest, users exert varying degrees of force on a peripheral. Factors such as cognitive stress or caffeine intake can cause micro-tremors-high-frequency,

low-amplitude oscillations in displacement telemetry [**stress_pressure**]. If the AP is set to 0.1mm and the user's tremor amplitude is 0.15mm, the system will register "ghost" inputs.

2.3.2 Neuromotor Fatigue Signatures. Extended sessions lead to fatigue in the *flexor digitorum superficialis* and *extensor digitorum* muscles [**fatigue_flexor, clicking_fatigue_ergonomics_2024**]. Fatigue manifests in two measurable telemetry changes:

1. **Velocity Decay:** The "snap-back" velocity (the speed at which a finger releases a key) decreases as the extensor muscles tire [**wrist_fatigue_genre_2025**].
2. **Resting Weight Shift:** As the flexor muscles lose tonicity, the resting weight of the finger often increases, leading to deeper "resting" displacement in the analog sensor [**heimhofer2024finger, 1hp_esports_ergonomics**].

2.4 Constraints of Offline Calibration (Scope Context)

This thesis focuses on offline calibration from depth telemetry. While continuous telemetry at high polling rates could theoretically be processed via on-device inference using quantized INT8 models on peripheral MCUs [**tinymc_mcu_inference**], hardware-in-the-loop deployment requires firmware-level access and strict latency budgets [**monaco_sok_2018**]. Consequently, on-device Edge ML is treated as future work, and all calibration pipelines in this study operate offline.

2.5 Signal Processing and Optimisation Foundations

The adaptive calibration pipeline draws on several established signal processing and machine learning methods. This section describes their mathematical properties; their problem-specific justifications appear in the Design chapter.

2.5.1 Savitzky-Golay Smoothing. Savitzky-Golay (SG) filtering fits a polynomial of degree p to each sliding window of W samples in a least-squares sense and evaluates the polynomial at the window

centre [savitzky_golay_1964]. Because the convolution coefficients are precomputed from the normal equations, the runtime cost is $O(n)$ regardless of window width, identical to a moving average. Crucially, the polynomial fit exactly reproduces moments up to degree p , so peak positions and edge inflection points in the signal are not distorted—a property absent from symmetric Gaussian or box filters, which both shift extrema toward the window centre.

2.5.2 Hermite Cubic Interpolation: Akima and Makima. Akima interpolation constructs a piecewise cubic Hermite spline whose tangent at each knot is a weighted average of the four adjacent finite differences, with weights inversely proportional to the absolute difference between consecutive slopes [akima_1970]. This weighting isolates the tangent estimate from outlier slopes: a single anomalous interval does not propagate ringing across the spline, unlike a natural cubic spline whose tangents are coupled globally. Akima does not enforce monotonicity, so it can represent non-monotone smooth curves. Modified Akima (Makima) adds a half-mean-slope regularisation term to the weight formula, preventing zero-weight degeneracy when two adjacent slopes are equal and opposite—a degenerate case that arises frequently in quantized sensor signals.

2.5.3 Covariance Matrix Adaptation Evolution Strategy. Although no longer the primary tuning strategy in this thesis, CMA-ES is retained as a useful comparative baseline for continuous threshold optimisation. To optimize physical hardware thresholds like AP and RT, we require black-box optimizers that handle continuous, correlated variables well. CMA-ES is a derivative-free evolutionary optimiser for continuous search spaces [hansen_cma_2016]. It maintains a multi-variate Gaussian distribution over candidate solutions and, at each generation, moves the distribution mean toward the best-performing candidates and adapts the full covariance matrix to reflect the local curvature of the objective landscape. This covariance adaptation lets the algorithm exploit correlations between parameters (such as the inter-dependent relationship between optimum AP and

RT points) and navigate elongated, ill-conditioned valleys that cause gradient-free methods with diagonal step sizes (e.g. Nelder-Mead or coordinate descent) to stall. CMA-ES converges in $O(n^2)$ function evaluations for an n -dimensional problem, far fewer than the $O(N^n)$ evaluations required by grid search over a width- N grid.

2.5.4 Bayesian Search and Threshold Sweeping. Bayesian optimisation provides a complementary black-box strategy when each objective evaluation is expensive. Rather than sampling candidate thresholds uniformly, it fits a surrogate model of the objective and selects the next trial by maximising an acquisition rule that trades off exploitation (promising regions) and exploration (uncertain regions). In Optuna, this process is commonly implemented with the Tree-structured Parzen Estimator (TPE), which models densities of good and bad configurations and proposes candidates likely to improve the current best value [optuna_akiba_2019, bergstra_tpe_2011]. Compared to population-based methods, Bayesian search is often more sample-efficient in low- to medium-dimensional threshold tuning, which is relevant when AP/RT candidate evaluation requires replaying long telemetry traces.

Threshold sweeping remains useful as a deterministic baseline and as a calibration tool for firmware constraints. In practice, sweeping can be configured as a coarse-to-fine procedure: a broad initial grid identifies stable operating regions, followed by local refinement around the best cells. This is especially valuable when simulating heavy firmware finite-state machines (FSMs), where each trial can involve high-frequency event replay, debounce/RT transition logic, and strict latency accounting. Under such cost profiles, exhaustive global sweeps become quickly intractable, while Bayesian search can reduce the number of full FSM simulations needed to reach competitive parameter settings.

2.5.5 Ensemble Methods: Random Forests and Gradient Boosting. For the offline ML pipeline, ensemble methods are particularly suited for extracting features from sliding-window time-series data be-

cause they are robust to scale differences and irrelevant features. A Random Forest [breiman_rf_2001] builds an ensemble of decision trees, each trained on a bootstrapped subsample with a random subset of features considered at each split, and averages their posterior class probabilities. The double randomisation decorrelates tree errors, yielding ensemble variance lower than any individual tree without significantly increasing bias. Gradient Boosted Trees [friedman_gb_2001] instead fit trees sequentially: each tree is trained to correct the pseudo-residual of the current ensemble via gradient descent on a differentiable loss. This staged correction reduces bias relative to a Random Forest, at the cost of longer training time and greater sensitivity to hyperparameter tuning.

2.5.6 Dilated Fully Convolutional Networks. While ensemble methods excel with hand-crafted features, deep learning models can automatically extract hierarchical representations directly from raw, unaligned telemetry data. Fully Convolutional Networks (FCNs) adapted for time-series employ stacked layers of 1D dilated convolutions. Dilated convolutions expand the temporal receptive field exponentially without requiring pooling layers that destroy temporal resolution, making them well-suited for capturing long-range dependencies in continuous analog sensor streams [bai2018tcn].

3. Related Work

This chapter surveys prior work across five domains that the proposed system draws on: adaptive threshold methods for HID devices, hyperparameter and threshold optimization, temporal sequence labeling from biosignals, programmatic labeling and weak supervision, and dilated convolutional architectures for sequence modeling.

3.1 Adaptive Threshold Methods for HID Devices

The problem of optimizing actuation parameters for human input devices has a long history in HCI, though prior approaches have relied on fixed heuristics rather than per-user learned models.

Kim et al. [impact2018] introduced *impact activation*, an alternative actuation scheme for mouse buttons in which a click is registered at the first

contact force peak following a rapid finger descent, rather than at a fixed displacement threshold. Their evaluation showed faster reaction times and fewer double-click errors under time pressure. Importantly, impact activation still relies on a single fixed heuristic (the first force peak), making it unsuitable for users with atypical contact dynamics, and it does not adapt to within-session fatigue. This thesis learns thresholds from the user’s own recorded telemetry rather than applying a globally fixed policy.

Trewin [dynamic_keyboard] proposed the *Dynamic Keyboard*, which automatically adjusts keyboard response parameters (minimum press duration, maximum bounce interval) to accommodate users with motor impairments, by observing an initial calibration sequence and adjusting parameters using rule-based adaptation. The Dynamic Keyboard targets binary on/off switch events and adjusts temporal debounce windows; its adaptation rules are domain-expert heuristics rather than models trained from user data, which was a natural design choice given that sub-millimetre analog depth sensing was not commercially available at the time of publication. The proposed system is complementary in motivation—both aim to reduce unintended actuations—but operates at a different signal level (continuous depth at 1 kHz vs. binary event timing) and uses a data-driven model rather than hand-coded rules.

Current commercial implementations [wooting_rapid_trigger, logitech_superstrike_2026] expose RT and AP as user-adjusted controls without closed-loop personalization, relying on subjective trial-and-error rather than recorded telemetry. The firmware-faithful threshold optimizer in this thesis treats RT/AP tuning as an explicitly optimized per-user objective, closing the loop that these products leave open.

3.2 Hyperparameter and Threshold Optimization

The broader question of how to search continuous parameter spaces with limited evaluation budgets has been extensively studied in the context of machine learning hyperparameter optimization (HPO). Bergstra et al. [bergstra_tpe_2011]

introduced the Tree-structured Parzen Estimator (TPE), a Bayesian approach that models the density of good and bad hyperparameter configurations and selects candidates likely to improve the current optimum. Optuna [optuna_akiba_2019] operationalizes TPE in a modern framework with dynamic search space definition, pruning of unpromising trials, and distributed execution. Compared to population-based strategies such as CMA-ES [hansen_cma_2016], TPE is more sample-efficient in low- to medium-dimensional spaces and is better suited to workloads where each trial is expensive—for instance, when evaluating a candidate AP/RT pair requires full-length replay of telemetry through a firmware finite-state machine (FSM) simulator.

The threshold optimization problem here differs from standard HPO in a few concrete ways: the search space is two-dimensional (AP and RT), each trial evaluation must replay telemetry through a faithful software replica of the peripheral’s debounce and state-transition logic, and the objective is a behavioral count (false actuations and missed clicks) rather than a differentiable loss. Optuna’s sampling algorithm is used directly; what this thesis adds is the domain-specific trial evaluator that sits inside the study.

3.3 Temporal Sequence Labeling in Biosignal Processing

Detecting click intent from continuous depth telemetry is an instance of temporal sequence labeling, a problem that has received substantial attention in the context of richer biosignals.

Surface electromyography (sEMG) is the closest related modality. Wang et al. [reactemg] presented ReactEMG, which uses masked autoencoder pre-training on sEMG to achieve stable, low-latency intent detection from eight-channel surface electrode recordings. Antuvan and Masia [lda_emg] applied Linear Discriminant Analysis to simultaneous classification of wrist and finger movements from sEMG, demonstrating real-time throughput on embedded hardware. Both systems operate on multi-channel electrode signals that encode rich neuromuscular information before mechanical actuation

occurs; their signal-to-noise advantage over single-sensor depth telemetry is substantial. The proposed system, by contrast, uses only the depth signal already present in the peripheral itself, requiring no additional sensing hardware. This imposes a harder classification problem—depth telemetry cannot observe pre-actuation motor intent—but removes the deployment barrier of wearable electrodes.

Keystroke dynamics literature [typeformer2022, continuous_keystroke_mdpi] models typing patterns for biometric authentication from binary key event timestamps. TypeFormer [typeformer2022] applies a Transformer architecture to sequences of inter-key timings and achieves high verification accuracy on mobile devices. These methods treat the keystroke as an atomic event and model the temporal pattern between events; they do not model sub-millisecond intra-click dynamics. The proposed system operates at a finer granularity—frame-level classification at 1 kHz—which requires a different feature engineering approach and exposes a different class of failure modes (contact-bounce artefacts, quantization noise) absent from binary event streams.

3.4 Weak Supervision and Algorithmic Bootstrapping

The geometric auto-labeler introduced in Section 4.2 fundamentally differs from programmatic weak supervision frameworks such as Snorkel [ratner_snorkel_2020]. Snorkel defines *labeling functions*—heuristic rules, distant supervision signals, or domain-expert logic—that each assign noisy labels to unlabeled examples, and combines them via a generative model that learns the accuracy and correlations of each labeling function. The combined label distribution is then used as the training signal for a downstream discriminative model.

Instead of deploying the geometric auto-labeler as an automated weak supervision signal for model training, the proposed system employs it strictly as an algorithmic bootstrapping assistant for human-in-the-loop annotation. As a result, the proposed system acts as a strongly supervised learning

pipeline accelerated by algorithmic pre-labeling, rather than a weakly supervised system relying on noisy heuristics. To account for natural human inconsistency in selecting the exact millisecond of an onset during this manual process, the pipeline also supports asymmetric soft labels (described in Section 4.4) that would encode physiological directional uncertainty into the verified targets rather than treating boundary annotations as rigid, perfect temporal observations; this objective was not used for the reported results, which train on hard labels, but is a potential avenue for future work.

3.5 Dilated Convolutions for Sequence Modeling

The OfflineFCN architecture adapts the dilated causal convolution structure introduced by van den Oord et al. in WaveNet [van den oord_wavenet_2016] for raw audio generation. WaveNet stacks dilated causal convolutional layers with exponentially increasing dilation rates to achieve a large receptive field with manageable parameter counts. The dilation schedule $d_i = 2^i$ allows the stack to cover $\sum 2^i$ steps of context with $O(\log N)$ layers rather than the $O(N)$ layers that a fixed-dilation architecture would require.

Bai et al. [bai2018tcn] conducted a systematic empirical comparison of Temporal Convolutional Networks (TCNs) and recurrent architectures (LSTMs, GRUs) across a range of sequence modeling benchmarks, demonstrating that dilated causal CNNs match or exceed recurrent models in accuracy while training faster and avoiding gradient vanishing. Their work established the TCN as a practical default for 1-D sequence classification, and the OfflineFCN design is motivated by the same receptive-field arguments they formalize. Ismail Fawaz et al. further benchmarked deep learning classifiers for time-series classification and found that FCN-based and ResNet-based architectures consistently outperform LSTM-based models on sensor data [ismail_fawaz_ts_2019], supporting the choice to build around convolutional blocks rather than recurrent ones.

The OfflineFCN departs from the standard causal TCN in one critical dimension: it uses symmetric (non-causal) padding rather than causal

(left-only) padding. This doubles the effective receptive field per layer for the same kernel size, since context is drawn from both directions—enabling a large symmetric receptive field, described in Section 4.4, to be achieved with only 8 blocks. The trade-off is that the resulting model cannot be deployed for streaming inference, since each output depends on future samples. This is appropriate for the offline analysis mode but excludes the model from the on-device path described as future work.

4. Design

This chapter details the architecture and algorithmic design of the proposed adaptive actuation system.

4.1 System Architecture

The system utilizes a multi-layered software and hardware stack designed to bridge high-frequency physical telemetry with a dynamic optimization engine. At the hardware layer, a modified PRO X2 Superstrike mouse streams continuous 120-level analog depth values at 1 kHz. This bypasses typical firmware constraints that generally limit reporting to mere 4-bit precision at 65 Hz and block reading after haptic events. The hardware communicates with the host operating system via an OS-level driver bridge that interfaces high-level prototyping environments with production-grade HID protocols. Sitting above this is a background sensor service that orchestrates and synchronizes the analog telemetry with OS events and other biometrics. Finally, a front-end orchestration client provides the user dashboard, manages recording state, and drives the downstream machine learning pipelines. The overall component topology is shown in Figure 4.1.

4.2 Data Labeling Methodology

Raw analog signals are inherently noisy. Before any label placement, the depth telemetry is smoothed using Modified Akima (MAKIMA) interpolation. MAKIMA was specifically chosen because it preserves signal monotonicity and avoids the severe overshoots characteristic of standard cubic splines, which is critical for accurate boundary detection

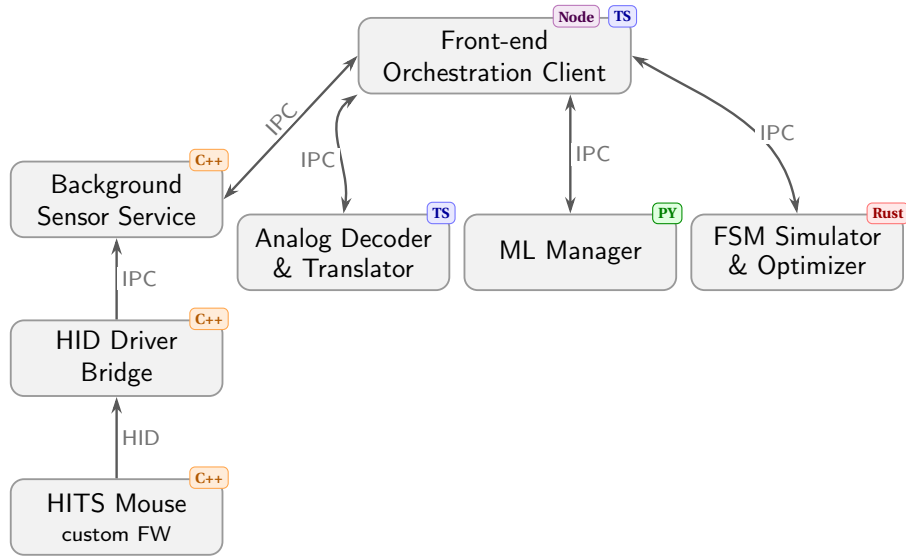


Figure 4.1: System architecture. Telemetry flows upward from a HITS-enabled mouse with custom firmware through a C++ HID driver bridge and a C++ background sensor service via IPC to the front-end orchestration client (Node/Electron + TypeScript). Three independent out-of-process back-end services communicate bidirectionally with the client via IPC: an analog decoder and translator (Python + TypeScript), a machine learning manager (Python), and a hardware FSM simulator and optimizer (Rust).

on a signal with sharp transitions.

4.2.1 Algorithmic Bootstrapping. To alleviate the bottleneck of manual annotation at scale, an automated heuristic pipeline pre-populates boundary candidates before human review. Detection anchors on the digital press/release event pair reported for each click to identify the approximate click window, then walks the smoothed analog signal within that window to locate the true biomechanical boundary:

- **Make Intention:** Starting from the click’s reported start, the algorithm walks backwards in time in search of the point of minimal analog depth, treating a run of frames without further improvement as evidence that the true rest position has been passed. This favors the deepest point actually reached over any single noisy sample, since sensor micro-tremors near rest position can otherwise produce spurious local minima that would misplace the boundary.
- **Break Intention:** Detected as the first point, walking forward from the click’s reported start, where

the (smoothed) rate of depth change falls below a small negative margin—i.e., the finger begins lifting with a speed larger than sensor noise. This typically precedes the reported key release and more accurately reflects the physical start of the finger lift.

This heuristic correctly handles the vast majority of clean, well-separated clicks, providing an accurate starting point for human reviewers to correct rather than annotate from scratch. Concrete search parameters and fallback behavior are implementation concerns, described in Section 5.2.2.

4.2.2 Iterative Human-in-the-Loop Annotation. Annotation followed an iterative three-phase strategy designed to maximize label quality while minimizing annotation effort:

1. **Algorithmic bootstrap.** The heuristic pipeline above generated initial boundary candidates for all recordings. Annotators reviewed these within a custom data visualization and labeling interface integrated directly into the front-end orchestration

client. The interface renders the raw 1 kHz analog depth signal alongside its derived velocity, and allows annotators to drag start and end boundary markers independently for the left and right button channels. Visual smoothing was applied on a per-recording basis to assist in distinguishing genuine kinetic events from micro-tremors. Annotators focused strictly on biomechanical intent, preserving deliberate presses including those not registered by the OS, while discarding involuntary micro-tremors and resting-weight drift. Approximately 50% of the dataset was verified and corrected in this phase.

2. **Model-assisted labeling.** Once the manually verified corpus was sufficiently large, a preliminary model was trained and used to auto-label the remainder of the dataset. Segments with a model confidence of 95% or above were accepted automatically, covering roughly two-thirds of each of the remaining unlabeled recordings.
3. **Mixed review.** The remaining clicks (primarily the edge cases) were handled by direct human annotation. A final manual review pass was conducted over all automatically labeled segments to validate quality before inclusion in the training corpus. Non-click segments (after trimming idle background) were manually verified to ensure that no labels were missed, with the help of the make and break probability curves produced by the model to highlight potential false negatives. This final review pass ensured that the dataset was as complete and accurate as possible, with a focus on edge cases and ambiguous signals.

The full extent and breakdown of the resulting annotated corpus is described in Section 5.1.

4.3 Data Preparation Pipeline

Raw recordings and their human-verified labels are transformed into normalized arrays shared by all three model architectures. Several design decisions govern this pipeline. First, onset boundary zones are widened by a small tolerance radius to absorb natural annotation imprecision, but are adap-

tively clamped against adjacent segment boundaries to prevent cross-segment label contamination. Second, derivative channels (velocity, acceleration, jerk, and resting-drift) are derived using zero-phase filtering to avoid introducing causal shift artifacts. Third, idle background periods are trimmed by an activity mask to reduce the dominance of background frames without discarding legitimate negative examples. Fourth, dataset splits are performed at the participant level so no individual’s recordings span both training and evaluation. Finally, normalization statistics are fitted on the training partition only, and statistics for kinematic channels are computed over active frames exclusively to prevent background mass from compressing the feature scale. Concrete implementation details—feature extraction windows, normalization constants, and weighting formulas—are described in Section 5.3.

4.4 Click-Intent Sequence Modeling

The click-intent problem is formulated as a per-frame four-class temporal segmentation task over continuous 1 kHz sensor streams. Each frame is assigned one of four mutually exclusive labels: Background, Make Onset, Click Hold, and Break Onset. Three model architectures are supported, positioned along a speed–accuracy trade-off curve suited to the realities of on-device, per-session deployment.

4.4.1 Random Forest. The Random Forest classifier operates on hand-crafted sliding-window features extracted from a local neighbourhood around each frame. It is the primary production model. Its main strength is training speed: a full session retrains in a few minutes on a consumer CPU. Feature importance scores are directly interpretable, exposing which kinematic cues drive the model’s decisions. The main weakness is a bounded temporal horizon limited to a few tens of milliseconds of context, which is sufficient for most click onset detection but may struggle on ambiguous edge cases where the intent signal unfolds over a longer transition. This architecture also fails to capture long-range dependencies and long context of user behavior. The pipeline also requires domain knowledge to design and cannot adapt to latent signal structure that

was not anticipated during feature engineering.

4.4.2 Gradient Boosted Trees. The Gradient Boosted Trees model shares the same feature extraction pipeline as the Random Forest but replaces the ensemble learner with sequential boosting, which iteratively focuses on the residuals of prior trees. Its key advantage is support for custom loss functions: a focal loss [lin_focal_2017] variant down-weights easy background frames, reducing over-confidence on the majority background class. Early stopping against a held-out validation metric limits overfitting. The weaknesses are slower training than the Random Forest, greater hyperparameter sensitivity, and the same fundamental constraint of a short-context hand-crafted feature representation.

A further design option explored for the GBT path is **asymmetric soft labels**, intended to address two simultaneous sources of asymmetry: natural annotator imprecision at boundary placement, and the directional difference in the cost of timing errors. For the Make Onset, a slightly *early* detection is acceptable—the finger is either already in motion or the signal is flat, so no threshold can trigger a click, and the error has negligible impact on the optimizer’s boundary estimate—whereas a *late* detection shifts the perceived onset forward in time, increasing input latency. For the Break Onset the direction of tolerable error is the same: a slightly *early* detection is benign because the signal is still rising or flat, and no break RT threshold can trigger a release on such a portion of the depth trace, whereas a *late* detection holds the click open beyond the true biomechanical release point, directly distorting the break boundary seen by the optimizer. The soft label generator encodes this asymmetry with a split-normal Gaussian centred on each onset frame: a wide left lobe and narrow right lobe for both Make and Break Onsets. This objective is implemented and supported by the training pipeline but was not used for the GBT results reported in this thesis, which are trained on hard labels with the focal loss objective described above.

4.4.3 Fully Convolutional Network. The dilated residual FCN operates directly on raw normalized

sensor streams without manual feature engineering. Stacked dilated convolutions grow the receptive field exponentially: with eight dilation stages and a kernel size of 7, the effective receptive field spans approximately 3 060 frames (≈ 3 s at 1 kHz), allowing the model to condition each frame’s prediction on long-range temporal context—pre-click muscle tension ramp-ups and post-click rebound dynamics that fall well outside any practical sliding window. The model is trained end-to-end on a three-channel Gaussian heatmap target, producing soft onset probability curves rather than hard class assignments. The trade-off is higher computational cost: the FCN requires a GPU or significantly longer CPU training time, is less interpretable than tree-based models, and benefits more strongly from larger datasets. It is therefore positioned as the high-accuracy architecture for offline post-processing, while the Random Forest handles latency-sensitive or resource-constrained deployment.

4.5 From Classification to Segmentation

The per-frame classifier produces a four-class probability array over the full recording. Converting these continuous probability outputs into discrete (start, end) click segments requires three sequential design choices: identifying candidate onset regions, electing a single representative frame from each region, and pairing each Make onset region with its corresponding Break onset region.

Candidate run detection. Two separate probability channels are extracted from the classifier output: the Make Onset probability and the Break Onset probability. Contiguous regions above a configurable threshold in each channel are identified as candidate runs. Make runs use a hysteresis-based entry/exit threshold pair rather than a single cut-off, so that a single physical press—whose probability hump can dip transiently below threshold due to quantization noise or softmax saturation—is not fragmented into multiple spurious candidates. Break runs are detected with a simpler thresholding scheme, since the Break probability curve is typically sharper and less prone to multi-modal fragmentation. The make and break thresholds are

hyperparameters that are jointly optimized with the onset frame election and pairing strategies described below.

Onset frame election. Within each candidate run, a single frame must be elected as the representative onset. Three strategies are evaluated, trading off robustness to probability-curve saturation against localization sharpness: **peak** selects the frame of maximum probability; **plateau median** instead returns the median frame among all near-peak frames, which is more stable when the curve saturates into a wide plateau; **steepest rise/fall** selects the inflection point of the probability curve, independent of any plateau. The optimal strategy and its associated threshold values are selected jointly via a Bayesian hyperparameter sweep (Optuna [optuna_akiba_2019]) over the validation set, optimizing the Intersection over Union (IoU) between predicted and ground-truth click segments.

Make-Break pairing. Given a set of elected Make and Break onset frames, a final matching step assigns each Make to exactly one Break. Three pairing strategies of increasing sophistication are considered. A **greedy first-fit** baseline pairs each Make with the first eligible Break in its search window. A globally optimal **assignment-based** strategy instead poses pairing as a minimum-cost bipartite matching problem over Make and Break confidence, guaranteeing no Break is claimed by more than one Make at the cost of a less scalable search. A **trained pairing scorer** further replaces the raw confidence-based cost with a learned classifier over pair-level features (confidences, timing, and local competition), targeting the cases where confidence alone is not a reliable indicator of the correct pairing. Because a globally optimal or learned strategy is not guaranteed to outperform the simpler baseline on every dataset, the three are compared empirically on the validation set, and the best-performing strategy is selected and stored alongside the model weights so inference uses identical pairing logic. Algorithmic and training specifics for each strategy are described in Section 5.4.

4.6 Parameter Optimization

After click-intent boundaries are extracted, the optimizer estimates the hardware threshold pair that best reproduces those boundaries under firmware-faithful trigger simulation.

4.6.1 Firmware State Machine Simulation. Each candidate threshold pair is evaluated with a frame-accurate simulation of firmware trigger logic.

In **simple-threshold mode**, the simulator is a 2-state machine: **press** is emitted when the make signal crosses the make threshold upward, and **release** is emitted when the break signal crosses the break threshold downward.

In **Rapid Trigger mode**, the simulator is a 3-state machine (Idle $\rightarrow$ Pressed $\rightarrow$ RT-Released) with rolling extrema. After actuation, **release** is emitted when depth falls by at least RT sensitivity below the running peak. From RT-Released, re-actuation occurs when depth rises by at least RT sensitivity above the running valley. The state resets to Idle only when the signal returns to zero, matching firmware reset semantics.

To preserve firmware behavior while keeping search efficient, signals are quantized before evaluation and searched on a discrete, hardware-relevant threshold grid.

4.6.2 Search Space and Objective. Optimization is discrete and exhaustive over the candidate threshold grid, yielding deterministic and reproducible solutions at exposed hardware resolution.

For each candidate, simulated click intervals are matched to labeled intervals using **overlap-constrained monotonic matching**. A label is matched only if at least one simulated interval overlaps that label interval. If multiple simulated intervals overlap the same label (oscillation/chatter), they are collapsed into one canonical matched interval using earliest make and latest break.

Let

- N = number of labeled clicks,
- TP = number of matched labels,
- FN = $N - TP$,
- FP = number of unmatched simulated intervals.

For matched pair i , define make and break boundary errors:

$$\begin{aligned} e_i^{(m)} &= t_{i,\text{sim}}^{(m)} - t_{i,\text{label}}^{(m)}, \\ e_i^{(b)} &= t_{i,\text{sim}}^{(b)} - t_{i,\text{label}}^{(b)}. \end{aligned}$$

Early errors are asymmetrically penalized with multiplier $\kappa \geq 1$:

$$\phi_\kappa(e) = |e| (1 + (\kappa - 1) \mathbf{1}[e < 0]).$$

Per-match timing distance is

$$d_i = w_{\text{make}} \phi_\kappa(e_i^{(m)}) + (1 - w_{\text{make}}) \phi_\kappa(e_i^{(b)}),$$

where $w_{\text{make}} \in [0, 1]$ controls make-vs-break priority.

Detection penalties are parameterized by base ms-equivalent detection cost c_{det} and recall/precision bias β :

$$c_{\text{miss}} = c_{\text{det}} e^\beta, \quad c_{\text{fp}} = c_{\text{det}} e^{-\beta}.$$

The final loss is

$$\mathcal{L} = \begin{cases} \infty, & TP = 0, \\ \frac{1}{TP} \sum_{i=1}^{TP} d_i + \frac{c_{\text{miss}} FN + c_{\text{fp}} FP}{N}, & TP > 0. \end{cases}$$

This yields interpretable controls: w_{make} sets make-vs-break timing emphasis, κ controls early-click severity, and (c_{det}, β) control overall detection strictness and recall-precision preference.

4.6.3 Pareto Frontier and Output. For each feature-pair configuration, the optimizer returns all evaluated candidates, the minimum-loss operating point, and a Pareto frontier over {error rate, average click distance} for interactive inspection.

Detailed serialization choices (grid layout, sentinels, and per-combination diagnostics) are implementation concerns and are described in Section 5.5.

5. Implementation

This chapter describes the software and hardware implementation of the system designed in Chapter 4.

5.1 Data Acquisition Campaign

The development and training of the machine learning backend required the construction of a robust ground-truth dataset spanning diverse ergonomic and biomechanical profiles. Data was collected from 69 participants via two separate campaigns: a controlled task dataset in-house, including both gamers and non-gamers, and a free-play dataset at the PolyLAN event. The two datasets were collected under different conditions to capture both structured and unstructured click dynamics.

5.1.1 Controlled Task Dataset (BIC Event). The first deployment cohort ($N = 26$) comprised a mix of competitive gamers to casual novices playing *Counter-Strike 2* (CS2). The protocol ran for approximately 40 minutes per user and was strictly structured to isolate specific click dynamics:

- **Configuration A/B Testing:** Following a uniform warmup phase, users played two 15-minute game blocks. The device hardware profile was hot-swapped halfway through the session, alternating between an aggressive configuration (AP at 1, RT at 1) and a highly conservative configuration (AP at 5, zero Rapid Trigger). The presentation order was randomized to mitigate sequence bias.
- **Weapon Rotations:** Within each 15-minute block, users were instructed to rotate their active weapon every five minutes. The arsenal was specifically chosen to evoke distinct motor patterns: the *USP-S* pistol for rapid individual tapping, the *M4A1-S* rifle for sustained tracking and spraying, and the *AWP* sniper rifle for slow, deliberate holding and release.

5.1.2 Free-Play Dataset (PolyLAN Event). The secondary dataset ($N = 43$) focused on unstructured free-play in a high-density LAN environment. Participants engaged in approximately 40-minute play sessions in their preferred title (CS2, *Valorant*,

or *League of Legends*). Similar to the controlled dataset, the hardware configurations were randomly (but uniformly) swapped from conservative to aggressive baseline profiles (and vice-versa) near the 20-minute mark to capture a wider range of organic responses. FPS players produced predominantly left-button click profiles consistent with primary-fire mechanics; *League of Legends* players generated a contrasting signature of high-frequency right-click interactions, characteristic of point-and-click action commands.

The aggregated dataset generated from these two campaigns ultimately produced tens of hours of raw 1 kHz telemetry across 69 participants, demanding the variance-based activity trimming described later in this chapter to isolate periods of genuine engagement before entering the classification pipelines.

5.1.3 Annotated Dataset Summary. Following manual annotation and variance-based trimming, the final curated dataset comprises 132 labeled recording files from 39 FPS-player participants, totaling approximately 12,556 seconds (~3.5 hours) of verified active telemetry. Left-channel annotations account for 39,088 labeled click cycles and right-channel annotations for 6,645 cycles, reflecting the predominance of primary-fire left-button interactions in FPS gameplay. The dataset was partitioned into training, validation, and test splits performed at the participant level to enforce strict subject isolation. We decided to hold out the *League of Legends* participants from the scope of this thesis, as their click dynamics were sufficiently distinct from the FPS cohort to warrant a separate study. The final dataset used for model training and evaluation consists of a total of 39 participants, using a split of 80% training, 10% validation, and 10% test. The participant-level split ensures that no individual appears in more than one partition, preventing data leakage and ensuring that model performance reflects generalization to unseen users.

5.2 Frontend and Telemetry Stack

The user-facing layer provides the orchestration dashboard, recording states, and test minigames. Telemetry ingestion is handled out-of-process by a

dedicated background sensor service, ensuring that 1 kHz hardware polling never stalls the frontend workflows.

5.2.1 Analog Decoder and Translator. Raw telemetry is stored as a sparse binary event stream capturing only state-change transitions. A dedicated decoder parses this format and resamples the event sequence to a uniform 1 ms grid using Modified Akima (MAKIMA) interpolation, which preserves the monotonic shape of the analog depth signal without the overshoot characteristic of standard cubic splines. Short resting gaps receive synthetic zero-value anchors before interpolation to prevent overshoot across idle periods. The output per frame consists of the quantized analog depth on a 0–120 scale, a digital press bit for each button, and a wall-clock timestamp. A client-side signal processing layer subsequently derives the kinematic channels—velocity, acceleration, jerk, and resting drift—from the interpolated depth, making them available for visualization and downstream annotation.

5.2.2 Labeling Interface. The annotation interface renders the decoded 1 kHz analog depth signal for both buttons alongside the derived kinematic channels, enabling annotators to judge biomechanical intent from the full kinematic context rather than raw depth alone. MAKIMA interpolation is applied at twice the native sampling rate on the client side to smooth the velocity overlay for visual clarity, without introducing causal shift. Annotators interact with segment boundaries via draggable handles: the start and end boundaries of each labeled region can be adjusted independently per channel, whole segments can be repositioned by body-drag, and new segments are created by dragging in unlabeled space. Visual overlays distinguish manual annotations from algorithmically generated candidates, and highlight model-generated suggestions alongside their associated confidence scores.

Annotation followed the three-phase iterative strategy described in Section 4.2. The bootstrapping heuristic (Section 4.2) is implemented as a pure TypeScript module shared between the interactive labeling UI and an offline batch labeler,

so both paths produce identical candidates from identical inputs. It operates on Savitzky-Golay-smoothed depth (order-2, odd window sizes from a precomputed coefficient table) rather than raw samples, since the raw 120-level stream is too noisy for a naive extremum search to be robust.

The Make boundary is found by a running-minimum search with patience: walking backward from the click's reported start, the algorithm tracks the lowest depth seen so far and stops once that minimum has not improved for 15 consecutive frames (or after a 200-frame search cap), returning the frame at which the minimum occurred. This is deliberately more conservative than a first-local-minimum search, since isolated microtremor troughs would otherwise terminate the walk prematurely. The Break boundary is found by walking forward from the same start point over the smoothed depth's finite-difference speed, returning the first frame where speed drops below -0.05 (in normalized depth units per ms); this hysteresis margin, rather than a literal zero-crossing, absorbs sensor noise near the peak. If no such frame is found before the click's reported end, the search falls back to that end index rather than leaving the boundary undefined. Both thresholds, along with the search window and patience length, are exposed as user-tunable parameters persisted across sessions, since their optimal values were found to vary slightly between annotators' preferred visualization settings.

This heuristic performs well on clean, well-separated clicks but has systematic failure modes that motivate the full annotation pipeline. It relies entirely on a detected digital event as an anchor, so clicks that were biomechanically executed but not registered by the firmware—because the actuation threshold was set deeper than the finger reached—are entirely invisible to it. The Make search remains sensitive to sustained noise or drift that shifts the running minimum away from the true rest position, while the Break search can be fooled by a slow, low-speed release that never crosses the hysteresis margin, silently falling back to the reported end frame. A separate post-hoc pass detects and trims boundary collisions when adjacent detected clicks overlap. These failure modes—unregistered presses,

drift-induced misplacements, and slow releases—are precisely the edge cases most informative for training a robust intent model, and are the primary reason human review remained indispensable throughout the annotation process.

5.3 Machine Learning Backend

The machine learning pipeline is encapsulated in a dedicated Python intent-detection backend built with scikit-learn, LightGBM [ke_lightgbm_2017], and PyTorch.

5.3.1 Loading and Frame Extraction. Each recording file is paired with a JSON label sidecar validated at load time. The 120-level analog depth stream is normalised to $[0, 1]$ by dividing by 120, and binary digital press signals are cast to $\{0.0, 1.0\}$. Each manually annotated click segment maps to three frame-level classes: Make Onset, Click Hold, and Break Onset. Onset zones extend ± 4 ms from each boundary marker to absorb natural annotation imprecision—a tolerance calibrated empirically to preserve signal sharpness while remaining robust to small annotator variability—and are clamped against adjacent segment boundaries to prevent cross-segment label contamination. A mtime-keyed in-memory cache prevents redundant re-parsing of unchanged files.

5.3.2 Preprocessing and Signal Augmentation. The analog channels optionally receive temporal smoothing (moving average, Gaussian, Savitzky-Golay, median, or EMA). Kinematic derivative channels—velocity, acceleration, jerk, and their absolute-value variants—are computed via zero-phase Akima-spline upsampling followed by centered finite differences and downsampling, eliminating causal shift. A resting-drift channel is derived as the difference of a fast and a slow exponential moving average, both applied zero-phase, capturing low-frequency postural drift independently of click dynamics. Continuous channels are optionally decimated using a zero-phase Chebyshev IIR low-pass filter before striding; discrete digital channels are strided only. A rolling-window activity mask based on configurable amplitude and variance thresholds trims inactive interior gaps, retaining short margins around each boundary and all labeled frames, and splits the recording into

independent active sub-sequences.

5.3.3 Splitting, Normalization, and Sample Weighting. Sub-sequences are partitioned into train, validation, and test at the sequence level. The preferred strategy is participant-level k-fold cross-validation: sequences are grouped by `user_id` (extracted from the file path via `p\d+` regex), each fold holds out exactly one participant group. Z-score normalization is fitted on the training partition only in order to prevent information leakage from the validation and test sets. For kinematic channels (velocity, acceleration, jerk, drift) the standard deviation is computed over non-background frames exclusively to prevent the background mass from compressing the feature scale. Binary digital and constant per-recording configuration channels are excluded from normalization.

Click-balanced per-frame sample weights equalize participant contributions. For participant u with C_u click events and b_u background frames, the weight of a frame belonging to click event e spanning $|F_e|$ frames is $(1/C_u)/|F_e|$, and the weight of a background frame is $1/b_u$. Weights are then rescaled per class to preserve the click/background mass ratio. The Random Forest additionally applies inverse-frequency class weights scaled by configurable per-class multipliers, providing an independent knob for the cost of each misclassification type, though this did not yield significant performance gains in practice.

5.3.4 Feature Extraction and Model Ingestion. For the Random Forest and Gradient Boosted Trees, sliding-window feature extraction produces one tabular vector per frame. Each window is centered on the target frame and computes a fixed bank of per-channel statistics: window mean, first-to-last slope, peak, velocity at the centre frame, time-to-peak, minimum, and time-to-trough. A set of cross-channel features is always appended: left/right analog correlation, whether each digital channel fires during an analog rise, and the maximum analog value across both switches. The feature set grows by one additional per-channel block for each derived channel enabled (e.g. velocity, acceleration, jerk, drift). An extended feature bank is also available,

adding higher-order statistics such as zero-crossing rate, TKEO energy, and kurtosis. Optional context probes append raw or locally averaged analog samples at configurable temporal offsets (e.g. ± 50 ms, ± 100 ms).

The FCN bypasses tabular extraction and receives raw $[C, T]$ tensors directly. The architecture consists of an input projection convolution, eight stacked dilated residual blocks each containing two convolutional layers, and a per-frame output classifier—18 convolutional layers in total. The dilation factor doubles at each successive block $(1, 2, 4, \dots, 128)$, producing an effective receptive field of

$$RF = 2(k-1)(2^D - 1)$$

which evaluates to 3 060 frames (≈ 3 s at 1 kHz) with kernel size $k = 7$ and $D = 8$ dilation stages. Each block applies symmetric non-causal padding—equal context on both sides of each convolution—so the model introduces no temporal shift in its predictions. A ReLU activation and dropout are applied between the two convolutional layers of each block; the skip connection within each residual block is an identity map throughout, as all blocks share the same hidden channel dimension. Training labels are converted to a 3-channel heatmap: channels 0 and 1 carry Gaussian probability bumps ($\sigma = 8$ frames) centred on each Make and Break onset frame respectively, and channel 2 carries a binary hold mask. Because onset frames are vastly outnumbered by non-onset frames within each channel—even after the click-balanced weighting of Section 5.3 equalizes participant and event contributions, the great majority of frames in any window are still confidently-negative background—standard binary cross-entropy would let these easy negatives dominate the gradient. We therefore use the focal loss of Lin et al. [lin_focal_2017], which multiplies the per-frame BCE term by a modulating factor that vanishes as the model’s confidence on that frame increases:

$$\begin{aligned}\mathcal{L}_{\text{focal}}(p, y) &= (1 - p_t)^\gamma \cdot \text{BCE}(p, y), \\ p_t &= yp + (1 - y)(1 - p)\end{aligned}$$

where p is the sigmoid output, $y \in [0, 1]$ the

heatmap target, and γ the focusing parameter ($\gamma = 2$, the default proposed by Lin et al.). As training progresses and background frames are classified with increasing confidence ($p_t \rightarrow 1$), their contribution to the loss is suppressed, concentrating gradient signal on the sparse, still-ambiguous onset frames. The loss is computed per channel per frame without reduction, then scaled by the per-frame sample weights from Section 5.3 before averaging, so participant- and event-balancing and difficulty-based re-weighting act jointly rather than one overriding the other. When sequence-window training is used, an 8-frame zero-weight gap is inserted between recordings in the concatenated hypersequence so no sliding window can span a boundary.

5.3.5 Gradient Boosted Trees. The Gradient Boosted Trees model shares the sliding-window tabular feature extraction pipeline described above; every per-frame feature vector is identical to the one supplied to the Random Forest. The distinguishing design concerns are the training objective and the label targets.

The pipeline additionally supports replacing hard one-hot labels at onset boundaries with asymmetric soft targets: a split-normal Gaussian centred on each annotated onset frame, with independent standard deviations on either side—for Make Onset, a wider left than right standard deviation encodes that detecting slightly early is less harmful than a late detection, and for Break Onset the asymmetry is reversed. This soft-labeling path is implemented but was not used for the reported GBT results; the models evaluated in Chapter 6 are trained on standard hard labels.

For the reported results, training uses the multiclass generalization of focal loss,

$$\mathcal{L}_{\text{focal}}(p_c) = -(1 - p_c)^\gamma \log p_c,$$

where c is the ground-truth class (Make Onset, Break Onset, Hold, or Background) and p_c the softmax probability the model assigns to it, for the same reason motivating its use in the FCN: background frames vastly outnumber onset frames, and standard cross-entropy would let the many easy, confidently-classified background frames domi-

nate the boosting gradient at the expense of the rare onset transitions. LightGBM does not expose focal loss natively, so it is supplied as a custom objective: the standard softmax cross-entropy gradient and Hessian are each scaled by a per-sample focal factor derived from p_c and γ , which attenuates both to near zero for confidently-correct frames and leaves them close to their unscaled values for hard or misclassified frames, concentrating tree splits on the latter. Sample weights are folded into this same custom objective, since LightGBM’s built-in dataset weights are not applied to gradients supplied by a custom objective function. Early stopping is evaluated on hard-label accuracy over the held-out validation partition. When the soft-label path is enabled instead, the one-hot target in the cross-entropy term is replaced by the split-normal soft targets described above, yielding a custom gradient and Hessian that align the objective with the continuous target distribution rather than a hard class assignment; this configuration was not exercised for the results in this thesis.

5.4 From Classification to Segmentation

The per-frame classifier produces a probability array over all four classes for the full recording. Converting these continuous outputs into discrete (start, end) click intervals requires three sequential stages: identifying candidate onset regions, electing a single representative frame from each region, and pairing each Make onset with a corresponding Break onset.

Segmentation is implemented as a single module shared, unmodified, between the offline training/evaluation pipeline and the online labeling daemon, with parity between the two paths enforced by dedicated regression tests; this guarantees that the strategy selected during validation behaves identically at inference time.

5.4.1 Candidate Run Detection. Make and Break run boundaries are located with a vectorized scan—computing the forward difference of a boolean above-threshold mask—rather than a per-frame loop, which replaced an earlier $O(T)$ while-loop implementation in the daemon with an equivalent scan over run boundaries only. The Make chan-

nel additionally applies a Schmitt-trigger hysteresis, opening a run when probability exceeds the entry threshold and closing it only once probability falls to half that value, which prevents a single physical press from fragmenting into multiple candidates when its probability hump dips transiently due to softmax saturation or quantization noise. Break runs use a single-threshold scheme, as the Break probability signal is generally sharper and less prone to mid-run fluctuations. Because the same make- and break-run scan is repeated at every point of the hyperparameter sweep described below, per-run onset frames are precomputed once per election strategy and cached, so re-evaluating a new threshold pair against an already-computed set of runs does not repeat the election computation.

5.4.2 Onset Frame Election. Three election strategies are implemented: **peak** (argmax over the run), **plateau median** (median index among frames within 0.5% of the run peak, falling back to the peak itself when the plateau collapses to a single frame), and **steepest rise/fall** (the frame of maximum probability gradient, with index 0 explicitly excluded from the search to avoid a finite-difference boundary artifact, and a fallback to the run’s first frame when the curve never falls). All three run in linear time over the number of candidate runs, and are vectorized to avoid per-run loops. The steepest-rise/fall strategy is the most robust to plateauing, but is sensitive to noise in the probability gradient; the plateau-median strategy is more robust to noise but can be biased toward the center of a long plateau; and the peak strategy is the simplest and fastest, but can be misled by spurious local maxima. The optimal strategy is selected via hyperparameter sweep on the held-out validation set.

5.4.3 Make-Break Pairing. Pairing is posed as an assignment problem over a dense $N_m \times N_b$ cost matrix, solved exactly via `scipy.optimize.linear_sum_assignment`; invalid (Make, Break) combinations are excluded from consideration with a large sentinel cost. This Hungarian-assignment path costs $O(N_m \times N_b)$ to build and is cubic to solve, which motivates keeping a **greedy first-fit** alternative—pairing each

Make with the first unclaimed eligible Break via a monotonically advancing pointer, $O(N_m + N_b)$ —for the largest concatenated evaluation sets. A **trained pairing scorer** (logistic regression over standardized features, fit with balanced class weights) replaces the raw confidence product as the assignment cost. Its six features are Make confidence, Break confidence, log inter-onset duration, Make run width, Break run width, and the number of competing Break runs within the Make’s search window. Critically, the scorer is trained only on *ambiguous* Makes—those with two or more competing Break candidates—since unambiguous cases carry no information about how to disambiguate. The reported training-set precision/recall for the scorer is therefore optimistic by construction and is not used to select the deployed strategy—instead, all three pairing strategies are compared against each other on held-out event-level F1, and the winner (with IoU as a tie-breaker) is the one stored and shipped.

5.4.4 Hyperparameter Selection via Bayesian Sweep. The optimal combination of election strategy, entry threshold, and exit threshold is determined by maximizing the segmentation IoU between predicted and ground-truth click intervals on the held-out validation set. A Bayesian search using a Tree-structured Parzen Estimator (TPE) sampler [bergstra_tpe_2011] is employed via Optuna [optuna_akiba_2019], backed by a persistent SQLite store that allows sweeps to be paused and resumed across sessions. When the Cartesian product of the discrete search space falls at or below 200 candidate combinations, the sampler automatically transitions to exhaustive grid enumeration to guarantee full coverage. The elected strategy, pairing strategy, and threshold values are stored alongside the trained model weights so that inference uses the identical configuration selected during validation.

5.5 Optimization Pipeline Integration

The parameter optimizer is implemented as a standalone Rust service that communicates with the front-end client via named-pipe IPC. It receives preprocessed analog signals and labeled click re-

gions from the ML manager, performs an exhaustive discrete search over candidate thresholds, and returns both best-solution and analysis artifacts as JSON outputs.

The core of the service is a frame-accurate simulation of firmware trigger logic, illustrated in Figures 5.1 and 5.2.

The deployed optimization path uses depth for both make and break channels, with candidate thresholds (T_m, T_r) where T_m is Actuation Point and T_r is Rapid Trigger sensitivity. In this path, the simulator runs a 3-state FSM (Idle $\rightarrow$ Pressed $\rightarrow$ RT-Released):

- Idle $\rightarrow$ Pressed if $m_i \geq T_m$ (emit make, initialize running peak),
- Pressed $\rightarrow$ RT-Released if $b_i \leq \text{peak} - T_r$ (emit break, initialize running valley),
- RT-Released $\rightarrow$ Pressed if $b_i \geq \text{valley} + T_r$ (emit make),
- RT-Released $\rightarrow$ Idle only when the signal returns to zero ($b_i \leq 0$ or valley ≤ 0), matching firmware reset behavior.

For non-depth break features, the backend also supports a simple 2-state threshold mode with candidate (T_m, T_b) :

- unpressed $\rightarrow$ pressed when $m_i \geq T_m$,
- pressed $\rightarrow$ unpressed when $b_i \leq T_b$.

Signals are quantized before evaluation so search remains firmware-faithful and efficient. In the deployed depth-depth path, thresholds are evaluated exhaustively over the exposed 10-level depth grid.

For each threshold candidate, simulated intervals are aligned to labels using overlap-constrained monotonic matching. The service computes timing and detection metrics (matched, missed, false positives, make-early, break-early, and latency summaries), evaluates the objective described in Section 4.6, and stores per-candidate simulated intervals and aggregate statistics.

Output artifacts include:

- the complete set of evaluated candidates,
- the minimum-loss solution,

- a Pareto frontier over {error rate, average click distance},
- landscape data for visualization.

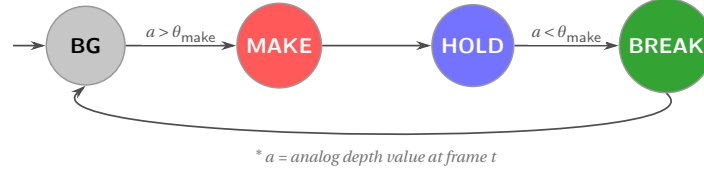


Figure 5.1: Simple threshold (no Rapid Trigger) FSM. A Make event fires when depth crosses above θ_{make} ; a Break event fires when depth falls back below the same threshold. The four states correspond directly to the four classifier output classes.

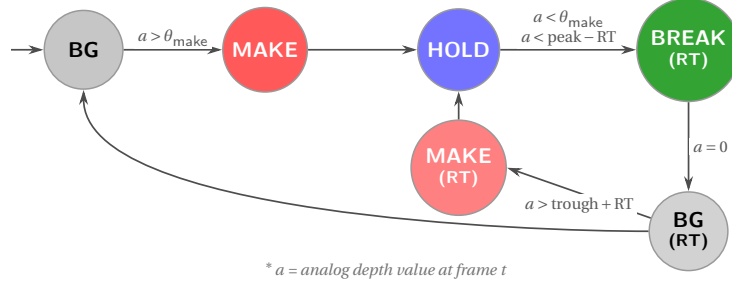


Figure 5.2: Rapid Trigger (RT) FSM. Once pressed, the button releases when depth drops by the RT sensitivity below the last observed peak. From the RT-released state the button re-actuates when depth rises above the last trough by the RT sensitivity, producing another Make onset; it fully resets to idle when depth returns to zero.

6. Evaluation

This chapter evaluates the system introduced in Chapter 5 along two complementary axes. Sections 6.1–6.4 evaluate the offline pipeline in isolation: the frame-level click-intent classifier and its classification-to-segmentation pipeline (Sections 6.2 and 6.3), and the firmware threshold optimizer built on top of it (Section 6.4), each against an appropriate baseline on held-out or full-corpus data. Section 6.5 then closes the loop with an end-to-end user study that tests whether the optimizer’s theoretical gains translate into measurable gameplay performance and subjective trust for real participants.

6.1 Experimental Setup

6.1.1 Dataset and Splits. All experiments use the curated corpus described in Section 5.1: 132 annotated recordings from 39 participants, yielding approximately 12,556,000 frames at 1 kHz. Separate models are trained for the left (L) and right (R) button channels, reflecting the large asymmetry in click volume (39,088 left-button cycles versus

6,645 right-button cycles). For each channel the data is split at the participant level into training (10,794,087 frames, L channel), validation (789,538 frames), and held-out test (972,663 frames) partitions. No participant’s recordings appear in more than one partition. Generalization is assessed with 5-fold grouped cross-validation, where each fold holds out a distinct group of participants (leave-one-participant-group-out), on the training and validation pool; the test set is evaluated exactly once per model after all hyperparameter selection is complete.

6.1.2 Baselines.

- **Majority-class baseline.** Assigns every frame the Background label. Provides the floor under severe class imbalance.
- **Heuristic bootstrap.** The OS-event-anchored walk-back detector described in Section 4.2, evaluated as a segmentor on the test set. Represents the pre-model ceiling achievable without a learned component.
- **Static default firmware configuration.** Actuation Point at level 5/10 (mid-travel), Rapid Trigger dis-

abled. Serves as the optimizer comparison point.

6.1.3 Metrics.

Detection and segmentation. A predicted click is counted as a true positive (TP) only if *both* its predicted make-onset and break-onset fall within a fixed tolerance $r = \text{DEPLOYMENT_EVAL_RADIUS_MS} = 4 \text{ ms}$ of the corresponding ground-truth boundary centers (the midpoints of the labeled make and break onset windows). Matching is one-to-one and greedy, performed via binary search over ground-truth make-onset centers. A predicted click that satisfies the make-onset radius but misses the break-onset radius (or vice versa) is *not* a TP: it is counted as both a false positive (the unmatched prediction) and a false negative (the unmatched ground-truth click). This dual-radius criterion is deliberately stricter than a single overlap or IoU threshold, since a click is only useful downstream if both of its boundaries are individually accurate. *Deployment F1* is the harmonic mean of click-level precision and recall under this criterion. *Mean IoU* is a separate, purely diagnostic Jaccard index computed only over matched (TP) pairs, and says nothing about clicks that failed the dual-radius gate; this is why IoU can remain high even when precision and recall are low (Section 6.6).

Boundary timing. For each matched pair we compute the signed make-boundary error $e^{(m)}$ and break-boundary error $e^{(b)}$ in milliseconds (positive = late, negative = early). We report the mean bias and the absolute spread (mean absolute deviation) per boundary. Boundary timing details are shown in Figure 6.1.

Cross-validation. For all three architectures (Random Forest, GBT, and FCN), 5-fold grouped cross-validation deployment F1 is reported as mean $\pm$ standard deviation, in addition to single held-out test set performance.

Optimizer objective and metrics. The firmware threshold optimizer (Section 6.4) scores each candidate (AP, RT) configuration with a loss function combining four terms: *TP rate*, the fraction of labeled click intents the configuration successfully

registers under the FSM simulator; *click distance*, the timing offset in milliseconds between the simulated firmware event and the labeled click; and *make latency* and *break latency*, the signed timing offsets in milliseconds between the simulated make/break firmware event and the corresponding labeled boundary. The loss combines these as $w_{\text{make}} \cdot e^{(m)} + (1 - w_{\text{make}}) \cdot e^{(b)}$ over matched clicks, with a fixed detection-cost penalty c_{det} applied per missed click, a break-timing weight β , and an exponent κ controlling the penalty curvature; all four are held fixed across sessions ($w_{\text{make}} = 0.6$, $c_{\text{det}} = 40 \text{ ms}$, $\beta = 0$, $\kappa = 2.0$) so that configurations are compared on a common scale. Statistical consistency of the optimizer’s improvement over the static default across sessions is assessed with a Wilcoxon signed-rank test on the per-session loss delta. The Wilcoxon test is chosen over a paired t -test because it makes no assumption of normally distributed deltas and is robust to the outlier sessions where the static default fails catastrophically (very large loss); it evaluates the null hypothesis that the paired loss differences are symmetric around zero, against the one-sided alternative that the optimized configuration’s loss is stochastically smaller.

User-study statistics. The end-to-end user study (Section 6.5) reports, per task and metric: the mean Δ (change from calibration to test round) for each group, a two-sided significance test between the experimental and placebo groups (p), the standardized effect size (Cohen’s d), and the observed post-hoc statistical power ($1 - \beta$) at $\alpha = 0.05$. Required sample size for a target power of 0.80 at the observed effect sizes is estimated with Lehr’s approximation ($n \approx 16/d^2$ per group).

Hardware. All timings are measured on an AMD Ryzen 9 7900X (12-core, 4.7 GHz), 16 GB RAM, with an NVIDIA RTX 5080 GPU. The Random Forest and GBT train on CPU; the FCN uses PyTorch with CUDA acceleration for both training and inference.

6.2 Classification and Segmentation Results

6.2.1 Deployment Performance on the Test Set. Table 6.1 reports click-level detection and boundary timing for all six release-candidate models on the

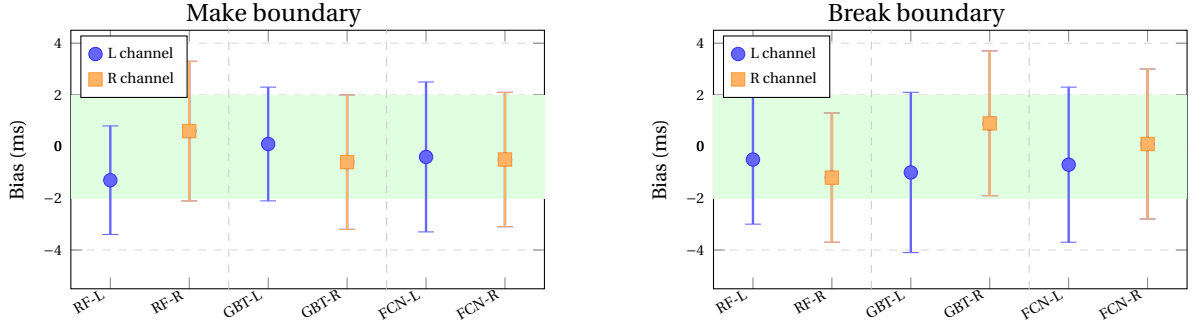


Figure 6.1: Make (left) and Break (right) boundary bias with absolute spread as error bars, for all six models. The green band marks a ± 2 ms reference tolerance. Positive bias = late boundary; negative = early. All models stay within ± 1.3 ms mean bias on both boundaries, with spread ≤ 3.1 ms. Dashed vertical lines separate the RF, GBT, and FCN model families.

held-out test set, alongside the heuristic bootstrap evaluated on the full 132-recording corpus (not just the held-out test set, since the heuristic has no train/test distinction to leak across and a larger, population-wide sample gives a more representative characterization of its performance). The Random Forest (L channel) achieves the highest deployment F1 of 97.0 %, consistent with its design-chapter positioning as the primary production architecture.

All models achieve sub-2 ms mean boundary bias, which is well within the perceptual threshold for click timing ($\approx 8\text{--}12$ ms). The heuristic bootstrap achieves a pooled deployment F1 of only 19.9 % on the left channel across the full corpus, with per-participant F1 ranging from 0.1 % to 75.0 % (Section 6.6). The dual-radius matching criterion (Section 6.1) means a click is only a true positive if both its make- and break-onsets are individually accurate; the heuristic’s duration estimate can still be accurate when a click is matched at all (IoU 91.6 %), but that says nothing about the many clicks it fails to match under the stricter dual-radius gate (see Section 6.6 for why this matched-pair IoU is comparable to the RF’s despite the large F1 gap). This gap — 19.9 % vs. 97.0 % for the RF — directly quantifies the value of the learned pipeline. The FCN underperforms significantly on the R channel (F1 83.6 % vs. 93.6 % for L), attributable to the much smaller right-button training corpus (6,645 cycles vs. 39,088) on which a 3-second receptive field

provides limited benefit. Figure 6.2 summarises deployment F1 across all three architectures and both channels. Figure 6.3 illustrates a correctly segmented click cycle, while Figure 6.4 shows a representative failure case where the predicted boundary drifts from the annotated onset.

6.2.2 Cross-Participant Generalization (5-Fold Grouped CV). Table 6.2 reports leave-one-participant-group-out cross-validation deployment F1 for all six models.

RF and GBT achieve near-identical CV means (93.2 ± 1.4 pp vs. 93.4 ± 1.0 pp on the L channel), supporting the design choice of RF as the default architecture. The FCN reveals a critical generalisation failure: its left-channel mean collapses to 59.7 ± 17.4 pp, with fold 4 dropping to 36.7 %. Despite achieving 93.6 % on the single held-out test set, the FCN does not seem to reliably generalise to unseen participant groups at current dataset sizes, confirming it is unsuitable for production use without a substantially larger training corpus. Figure 6.5 plots the per-fold deployment F1 for the RF and GBT models across all five folds: the L-channel models outperform their R-channel counterparts in every fold, reflecting the larger left-button training corpus.

6.2.3 Training Dynamics. Figure 6.6 shows per-iteration training and validation loss for the GBT models and per-epoch loss for the FCN models. The Random Forest is excluded as it is a single-pass

Model	Ch.	F1	P	R	IoU
Heuristic	L	19.9 %	20.0 %	20.0 %	91.6 %
Heuristic	R	12.2 %	11.5 %	12.9 %	89.9 %
RF	L	97.0 %	97.5 %	96.4 %	91.6 %
RF	R	91.8 %	93.0 %	90.5 %	90.0 %
GBT	L	93.7 %	93.2 %	94.3 %	89.9 %
GBT	R	91.3 %	91.3 %	91.3 %	91.7 %
FCN	L	93.6 %	92.3 %	94.8 %	91.5 %
FCN	R	83.6 %	86.2 %	81.1 %	91.5 %

Table 6.1: Click-level deployment metrics. The heuristic bootstrap is evaluated on the full 132-recording corpus (39 participants); the Random Forest, GBT, and FCN are evaluated on the held-out test set only. Best value in each column among the learned models is **bolded**.

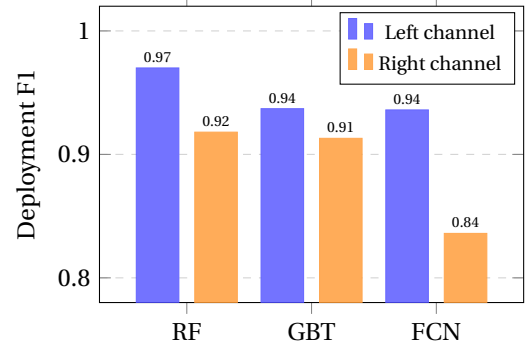


Figure 6.2: Deployment F1 by architecture and button channel on the held-out test set.

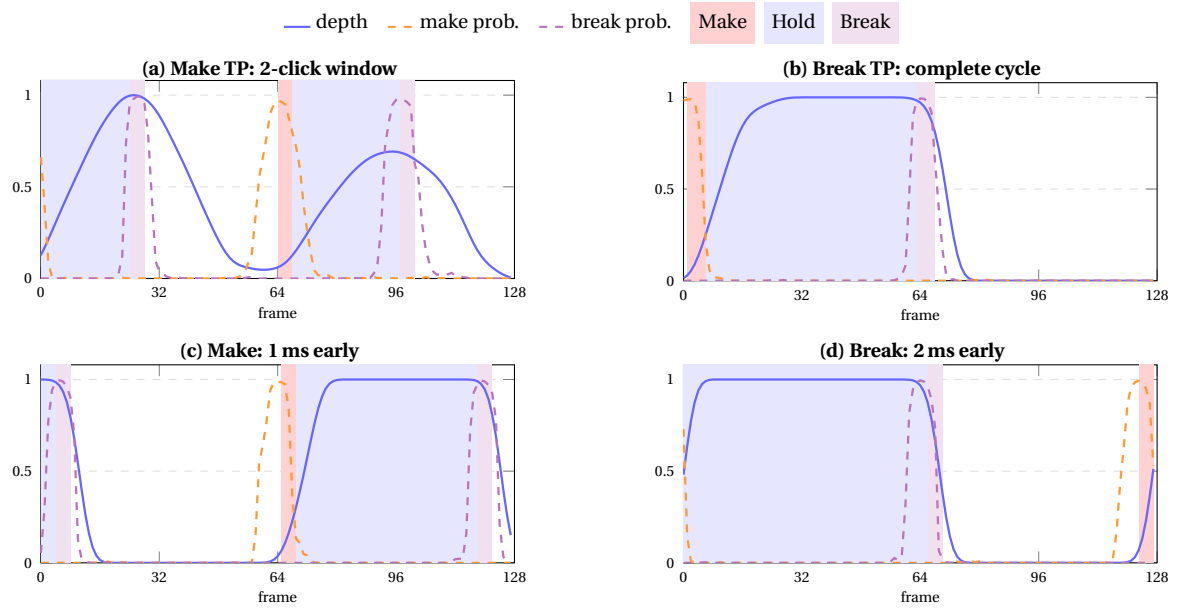


Figure 6.3: Correct and near-correct outputs (RF-L test set).

Model	Ch.	Mean F1	Std	Per-fold F1s
Random Forest	L	93.2 %	1.4 pp	[93.9, 95.3, 92.9, 91.0, 93.2] %
Random Forest	R	89.0 %	2.1 pp	[87.5, 90.6, 86.4, 88.2, 92.2] %
GBT	L	93.4 %	1.0 pp	[93.8, 94.7, 93.4, 91.8, 93.3] %
GBT	R	87.3 %	2.8 pp	[85.6, 89.5, 85.9, 83.9, 91.4] %
FCN	L	59.7 %	17.4 pp	[82.2, 48.1, 54.4, 36.7, 77.4] %
FCN	R	76.0 %	9.0 pp	[80.5, 82.1, 61.2, 70.2, 85.8] %

Table 6.2: 5-fold grouped cross-validation deployment F1, with each fold holding out a distinct group of participants.

batch learner with no iterative loss history. The GBT validation-training gap is substantially wider on the R channel than on the L channel, reflecting the harder right-button class distribution; in

the FCN panels, the model enters the overfitting regime past the marked best-validation epoch.

Figure 6.7 quantifies the corresponding

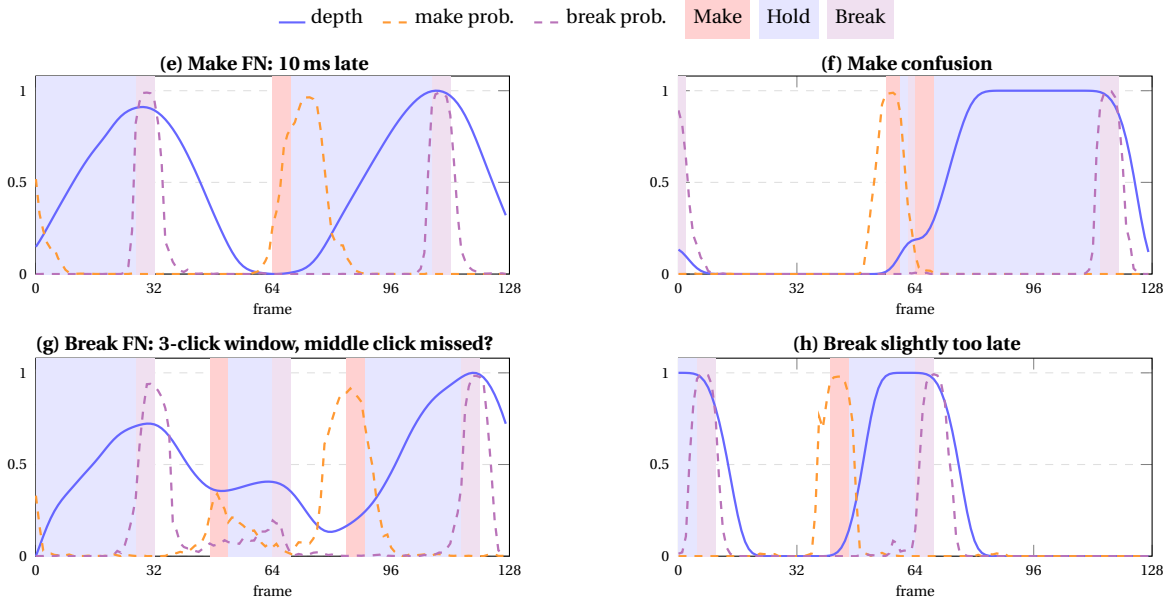


Figure 6.4: Failure modes (RF-L test set).

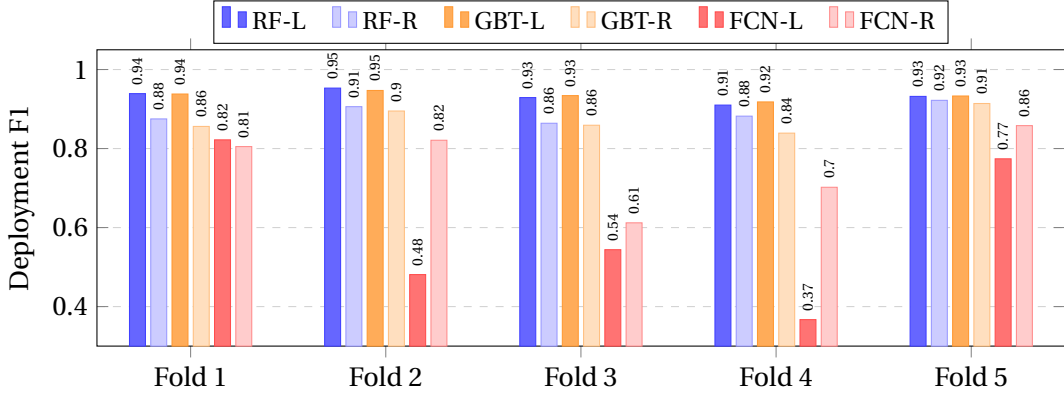


Figure 6.5: Per-fold grouped cross-validation deployment F1 for all six release-candidate models. The FCN’s collapse in folds 2–4 sets the extended vertical span.

accuracy-based generalisation gap for all six models. All gaps stay within $\pm 6\%$, so none of the architectures overfits severely at the accuracy level; the right-channel FCN shows the largest negative gap (-5.3%), consistent with strong dropout regularisation on a small corpus, while GBT and FCN left-channel models show small positive gaps ($< 1\%$), indicating marginal overfitting on the more abundant left-button training set.

6.3 Segmentation Configuration

6.3.1 Optimal Thresholds and Strategies. Table 6.3 shows the segmentation configuration selected by the Bayesian sweep for each model.

6.3.2 Pairing Strategy Comparison. The three pairing strategies (greedy first-next, Hungarian assignment, and trained logistic scorer) produce nearly identical segmentation IoU across all models and channels, with a maximum gap of 0.1 pp IoU. Once the classifier yields well-calibrated probability outputs the assignment problem is largely trivial, and greedy first-next is selected as the default for RF

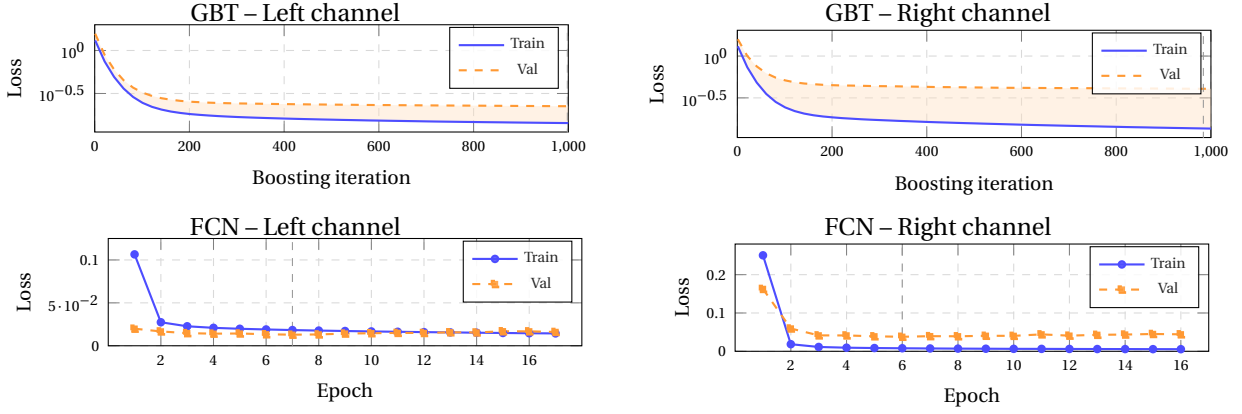


Figure 6.6: Training curves for GBT (top row, log-scale loss over 1,000 boosting iterations, sampled every 20) and FCN (bottom row, focal loss per epoch). The dashed vertical line marks the best-validation checkpoint: iteration 998 (GBT-L), 984 (GBT-R), epoch 7 (FCN-L), and epoch 6 (FCN-R). The shaded band in the GBT panels shows the generalization gap (val – train loss).

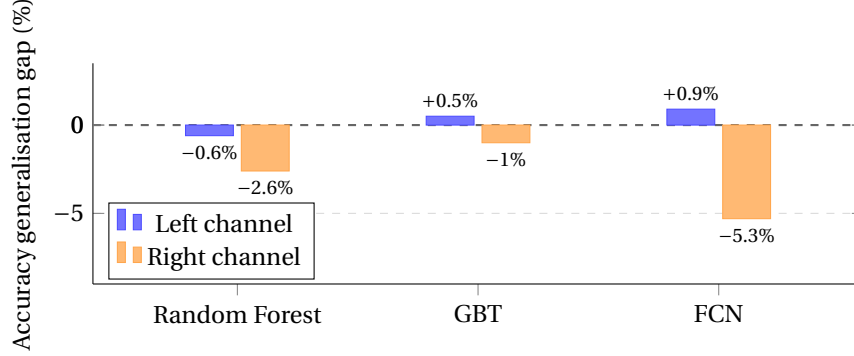


Figure 6.7: Accuracy-based generalisation gap $(a_{\text{train}} - a_{\text{test}}) / a_{\text{train}} \times 100\%$ for all six models, grouped by architecture. A positive value indicates higher training than test accuracy (overfitting); a negative value indicates the reverse.

Model	Ch.	θ_m	θ_b	Make method	Break method	Pairing
Random Forest	L	0.75	0.85	plateau_median	plateau_median	first_next
Random Forest	R	0.80	0.75	peak	peak	first_next
GBT	L	0.90	0.05	peak	plateau_median	scored
GBT	R	0.90	0.65	peak	plateau_median	scored
FCN	L	0.50	0.50	plateau_median	plateau_median	first_next
FCN	R	0.30	0.50	peak	plateau_median	hungarian

Table 6.3: Optimal segmentation configuration per model, selected by Bayesian sweep on the validation set. θ_m, θ_b : Schmitt-trigger entry thresholds on the probability output.

and FCN L, while the trained scorer is selected for GBT and Hungarian for FCN R.

6.3.3 Channel and Probe Ablation (Random Forest). Table 6.4 reports an additive ablation from window statistics only (no probes, no kinematics) through

to the full production feature set. All configurations use the same RF-L hyperparameters; only the pre-processing flags change.

Context probes contribute +2.17% deployment F1, roughly three times the combined gain of all

Configuration	Features	F1	Δ (row)	Δ (cumul.)
Window statistics only (no probes, no kinematics)	21	94.21 %	—	—
+ Context probes (± 10 ms, 5+5)	41	96.38 %	+2.17%	+2.17%
+ Velocity	59	96.97 %	+0.59%	+2.76%
+ Acceleration	77	97.02 %	+0.05%	+2.81%
+ Jerk	95	97.06 %	+0.04%	+2.85%

Table 6.4: RF-L deployment F1 under progressive feature inclusion. Δ (row) is relative to the previous row; Δ (cumul.) is relative to window statistics only. Context probes: 10 raw depth samples at ± 10 ms offsets (5 left, 5 right).

kinematic derivative channels (+0.68%). The 10 raw depth samples at ± 10 ms give the RF direct access to the local depth trajectory, allowing it to approximate velocity and acceleration implicitly from adjacent probe values; explicit kinematic channels are therefore largely redundant, and jerk adds a negligible +0.04% on top.

Figure ?? shows how importance is distributed across channel groups at each ablation step. `analog_L` — which subsumes both window statistics and probe values — is the dominant group in every configuration. When kinematic channels are absent its share reaches 70.6 % (with probes) or 81.9 % (window statistics only). As kinematics are added, velocity absorbs part of that share but `analog_L` never falls below 31 %, confirming that the raw depth trajectory carries information that derived channels cannot fully substitute. `digital_L` remains stable at 18–29 % throughout, reflecting the independent predictive value of the OS-reported press event.

For the RF architecture, probe range and density therefore matter more than channel engineering. The natural next ablation is a *wider-probe* study (e.g. ± 50 ms or ± 100 ms offsets) to determine whether extending the 100 ms probe horizon further narrows the gap to the FCN’s 3 s receptive field.

6.4 Optimizer Evaluation

6.4.1 Comparison with Static Default. The firmware threshold optimizer is evaluated by comparing the per-user optimal (AP, RT) configuration against the device’s static default (AP = 5/10, RT disabled) across all 238 channel-records from 132 sessions and 39 participants — the full annotated corpus, not limited to the held-out test set. The optimizer

is not trained on any data split; it exhaustively evaluates the (AP, RT) grid for each session by replaying the recorded analog signal through the firmware FSM simulator, so evaluating on the full corpus does not leak test-set information used to select model hyperparameters. For each session and channel, the optimizer exhaustively searches the 10-level (AP, RT) grid using the RF model’s segmented click intervals as labeled intent, applying the production objective function ($w_{\text{make}} = 0.6$, $c_{\text{det}} = 40$ ms, $\beta = 0$, $\kappa = 2.0$). The static default is scored through the identical firmware FSM simulator to ensure a fair comparison.

Table 6.5 summarises the comparison. The optimizer reduces loss by 49 % (mean) and increases the TP rate from 90.9 % to 97.5 %, recovering an additional 6.6 percentage points of labeled click intent. The improvement is consistent across all 238 channel-records: a Wilcoxon signed-rank test on the per-session loss delta yields $W = 0$ ($n = 238$, $p \ll 0.001$), confirming that the optimized configuration never performs worse than the static default on any session in the corpus. The most practically significant result is the make-latency reduction: the static mid-travel AP (level 5 of 10) requires the user to press well beyond the biomechanical onset before the firmware registers the event, producing a mean make latency of 27.0 ms. The per-user optimized AP reduces this to 15.4 ms — an 11.6 ms improvement that spans roughly one perceptual just-noticeable difference for actuation timing (≈ 8 –12 ms) and represents a directly perceptible responsiveness gain.

Break latency improves from a mean of 29.1 ms to 11.2 ms and a median of 21.5 ms to 9.0 ms, with no degenerate RT selections observed: setting the

Metric	Static default		Optimized		Δ	
	mean	median	mean	median	Δ mean	Δ median
Optimizer loss	36.2	28.4	18.4	15.3	-17.8	-13.0
TP rate	90.9 %	94.9 %	97.5 %	99.6 %	+6.6 pp	+4.7 pp
Click distance (ms)	27.8	23.6	14.8	12.7	-13.0	-10.9
Make latency (ms)	27.0	24.5	15.4	13.1	-11.6	-11.4
Break latency (ms)	29.1	21.5	11.2	9.0	-17.9	-12.5

Table 6.5: Optimizer comparison: static default versus per-user optimized (AP, RT) configuration. Mean and median over 238 channel-records (132 sessions, 39 participants).

detection-cost parameter to $c_{\text{det}} = 40$ ms removes the optimizer’s incentive to trade break-timing accuracy for a marginal TP-rate gain by maxing out RT hysteresis, so break latency is penalized consistently across all 132 sessions.

6.5 End-to-End User Study

The evaluations above quantify the pipeline’s internal correctness (detection accuracy, boundary timing) and the optimizer’s theoretical gain against the FSM simulator. Neither establishes that a per-user optimized configuration is perceptible or beneficial to an actual player. We therefore ran a controlled, double-blind user study to test whether the optimizer’s recommendation improves measured gameplay performance and whether users perceive a difference in trust toward the recommended configuration.

6.5.1 Protocol. 26 participants were recruited and randomly, evenly split into an experimental group (calibrated configuration applied, $n = 13$) and a placebo group (calibration game played identically, but no configuration change applied, $n = 13$), following a double-blind design. Group sizes were fixed in advance. Each session lasted 10–15 minutes and consisted of an onboarding phase (comfort settings, DPI, and a free run to familiarize the participant with the minigames) followed by three calibration-and-test rounds, one per task:

- **Flicking** (1-minute calibration game, then one test round): fast target-acquisition task, evaluated on targets hit and shot accuracy.
- **Tracking** (1-minute calibration game, then one test round): sustained cursor-on-target task, evaluated on kill count and fraction of time spent

on-target.

- **CPS** (10-second calibration game, then one test round of ≤ 30 s): clicks-per-second task, evaluated on raw click rate.

To ensure uniform task difficulty across participants and reproducible comparisons within each participant, target spawn locations and order were generated using deterministic seeded randomization rather than independent random sequences for each player. A single fixed seed was used across all 26 participants for each task, ensuring calibration and test rounds are directly comparable both within and across individuals.

Block-randomized target generation. The sequence generator uses block randomization to control target difficulty: targets are grouped into distance “buckets” (five equally spaced distance values from 200 px to 600 px for flicking and tracking), and within each conceptual block of b targets, each bucket distance is used exactly once, in a shuffled order. Formally, the generator partitions the maximum expected target count (240 targets per task) into $\lceil 240/5 \rceil = 48$ blocks of five targets each. Within every block, the five bucket indices are shuffled using the same seed’s pseudo-random stream, then the shuffled block is appended to the sequence. Any participant who completes $k \cdot 5$ targets in the time limit (where k is an integer) encounters exactly k complete blocks, thus exactly k targets at each distance, guaranteeing equal difficulty distribution regardless of task duration or participant performance. For participants who finish a partial block, the shortfall is at most one target per bucket, bounding the per-bucket imbalance.

Controlling seed effects across per-participant

calls. Rather than re-initializing the generator identically for both rounds (which would require re-running the same random number stream and risk subtle drift due to arena-size differences), the calibration round recorded the sequence of distance, angle, and speed values for every target placement. The test round then replayed this recorded sequence in reversed order, ensuring the exact same targets (at the same distances and angles) reappear but traversed backward. This reverse-replay approach: (1) eliminates re-seeding as a source of error by recording ground truth; (2) provides a fresh test sequence that is geometrically equivalent but not identical (avoiding simple memorization while preserving difficulty balance); and (3) preserves recorded speed samples from tracking calibration, so evaluation rounds can reuse them rather than drawing new samples that would pair each target’s distance with an uncorrelated speed.

Within-participant, holding the target geometry constant (same positions and distances, only in reverse order) isolates performance change as a measurement of true skill or calibration effect. Across participants, the global seed ensures the pool of reachable target positions and distance buckets are uniform, so inter-participant comparisons are not biased by one participant facing systematically easier or harder geometry.

To guarantee fidelity between calibration and test sequences, all target placements were pre-generated for all 26 participants before the study began, and each sequence was inspected to ensure no distance-clamping or angle-fallback events occurred. This pre-generation allowed us to verify that every target in both the calibration and reversed test sequence was placed cleanly within the configured distance buckets without any defensive fallbacks, eliminating any divergence between rounds due to arena-size variation or outlier positions. The pre-generated sequences were then loaded at runtime for each participant, ensuring exact reproducibility and uniform difficulty across all 26×3 task instances.

Per-participant seed variance. Although a global seed was fixed across all participants to enable cross-participant comparison, each participant’s

internal gameplay is still subject to stochasticity in mouse movement, reaction time, and click latency — factors that are system properties, not controlled in the protocol. The seeding of target positions is held constant to isolate the effect of the device calibration, leaving individual behavior variance as the independent variable of interest.

No warmup round preceded the timed tasks. Each task’s score is analyzed independently; the protocol does not attempt a single unified performance metric across tasks, since each task acts as a proxy for a distinct game context (FPS flicking, tracking-heavy shooters, and rhythm/CPS-bound inputs) rather than a single underlying skill.

6.5.2 Results. Table 6.6 summarizes the per-task comparison between the experimental (calibrated) and placebo groups. Effect sizes are reported as Cohen’s d ; $1 - \beta$ is the observed post-hoc power at $\alpha = 0.05$.

CPS. The calibrated group improved click rate by +10.7% against a −2.5% regression in the placebo group, a statistically significant and large effect ($p = 0.035$, $d = 0.92$). The median change is more modest (+2 clicks vs. −1 click, i.e. +3.2% vs. −2.0% of each participant’s own baseline), indicating the mean is pulled upward by a subset of participants with especially large gains rather than reflecting a uniform shift across the group; the direction of the effect nonetheless agrees with the mean. No misclicks were reported in either group on this task.

Tracking. No metric reaches significance at $\alpha = 0.05$, but the direction is consistently favorable: kill count and time spent actively clicking on-target both trend upward more for the calibrated group than for placebo. Given the sample size, these results are hints rather than conclusive evidence.

Flicking. No metric reaches significance. The calibrated group shows a small decrease in targets hit and in accuracy relative to placebo, which we interpret cautiously: at this sample size a −0.6 vs. +1.5 percentage-point accuracy difference is plausibly noise, but it is also consistent with a plausible mechanism (Section 6.6) in which a lower actuation point trades click-timing margin for accuracy

Task	Metric	Experimental Δ (mean / median)	Placebo Δ (mean / median)	Mean % Δ (exp. / placebo)	p (d , $1 - \beta$)
CPS	Clicks/s	+8 / +2	-2 / -1	+10.7% / -2.5%	0.035 ($d = 0.92$, $1 - \beta = 0.62$)
Tracking	Kills	+1 / +1	+0 / 0	+3.4% / +0.7%	0.068 ($d = 0.75$, $1 - \beta = 0.45$)
Tracking	On-target	-0.4% / +0.2%	+0.3% / +0.3%	-0.7% / +0.8%	0.310 ($d = -0.41$, $1 - \beta = 0.17$)
Tracking	Clicking on-target	+3.1% / +3.6%	+0.9% / +0.5%	+4.2% / +1.1%	0.114 ($d = 0.65$, $1 - \beta = 0.36$)
Flicking	Targets hit	+2 / +2	+5 / +4	+2.7% / +5.1%	0.073 ($d = -0.74$, $1 - \beta = 0.44$)
Flicking	Accuracy	-0.6% / -1.0%	+1.5% / +1.6%	-0.5% / +1.7%	0.188 ($d = -0.53$, $1 - \beta = 0.26$)

Table 6.6: User study results, experimental ($n = 13$) vs. placebo ($n = 13$). Δ columns report both the mean and median change from calibration to test round; the median is included alongside the mean because several rows (notably CPS) have means skewed by a small number of participants. Mean % Δ is the mean percent change relative to each participant’s own calibration-round baseline. Misclicks are reported as participant count out of 13 who registered at least one misclick.

in fast flicking tasks. Misclicks were reported by 3 of 13 calibrated participants on this task versus 0 of 13 placebo, though the study was not powered to treat this as conclusive.

Gamer-attribute interaction. A mixed ANOVA (group $\times$ gamer status, self-reported) tested whether self-identified gamers and non-gamers responded differently to the intervention. No significant group $\times$ gamer interaction was found on any task, indicating the calibration’s effect (or lack thereof) did not differ measurably between the two subpopulations at this sample size.

6.5.3 Recommendation Behavior. Independently of task performance, the optimizer produced a healthy spread of per-user actuation-point recommendations across the 26 participants (left-channel mean 22.6 of 120 depth levels, standard deviation 17.1), rather than collapsing to a single value or to the static default. This confirms that the models remained well-behaved when driving live per-user recommendations end-to-end, outside the offline evaluation setting of Section 6.4.

Figure 6.8 breaks this spread down by game scenario. Flicking and tracking calibration rounds converge to broadly similar distributions, peaking at 24 and 36 respectively (median 24 and 36) with no participant recommended above 48. CPS calibration is markedly more concentrated at the lowest level (16 of 26 participants at AP 12, median 12) yet is the only scenario with participants recommended at 60 and 72.

We attribute this high-AP tail to a behavioral artifact specific to the CPS task rather than to noise in the optimizer itself. Once rapid trigger (RT) engages

after the switch has crossed the actuation point once, subsequent re-clicks are driven by the release-then-cross reactive-trigger sensitivity rather than by the actuation point, which is only consulted for the *first* click of the sequence. Participants who anticipate the CPS prompt and pre-press before the round starts (a plausible strategy for maximizing raw click count within the 10-second window) generate a calibration signal in which the actuation point only ever matters for one make event, leaving the optimizer with a single, low-information point to fit against. This produces recommendations that are effectively under-constrained and free to drift toward the depth extremes, rather than genuinely indicating that a high actuation point benefits those participants.

Recommended reactive-trigger (RT) sensitivity, by contrast, showed almost no spread: 25 of 26 participants were assigned the lowest available level (6), with a single participant at the next level (12). Unlike the actuation-point recommendation, RT sensitivity did not differentiate participants in this study, which may reflect either a genuinely uniform optimum across users or limited sensitivity of the underlying signal at this stage of the pipeline.

6.5.4 Study Limitations. The study was underpowered for most secondary metrics: observed power ($1 - \beta$) ranges from 0.17 to 0.62 across non-CPS metrics, well below the conventional 0.80 target, so null results for Tracking and Flicking should be read as inconclusive rather than as evidence of no effect. Applying Lehr’s approximation to the observed effect sizes indicates that reaching adequately powered comparisons (target power 0.80 at $d \approx 0.75$) would require approximately 29 partic-

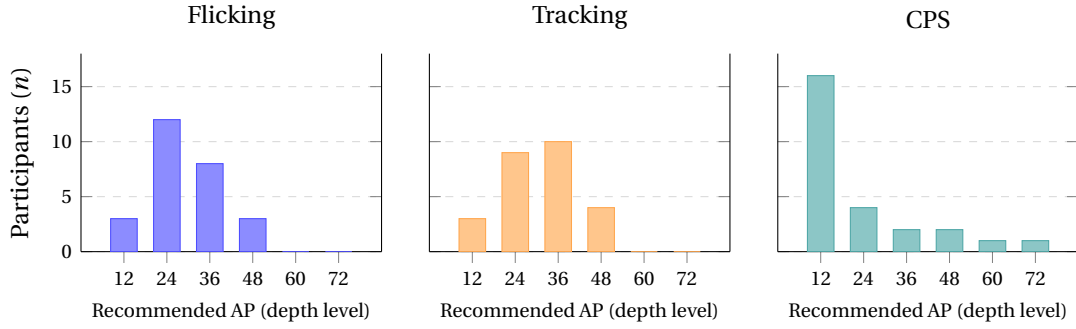


Figure 6.8: Distribution of per-participant left-channel actuation-point recommendations ($n = 26$) by game scenario, binned at each of the six reachable depth levels. Flicking and tracking both peak at 24–36 with no participant recommended above 48. CPS is the most concentrated at the lowest level (16 of 26 participants at AP 12) but also the only scenario with participants recommended at 60 and 72; Section 6.5 discusses the likely cause of this tail.

ipants per group (58 total), roughly $2.2\times$ the current sample. Between-subject group assignment also left substantial unmeasured skill-level variance uncontrolled, a single un-warmed-up test round per task leaves the results susceptible to a learning-effect confound between the calibration round and the test round, and gamer/non-gamer counts were not balanced in advance across groups (8/13 vs. 9/13 gamers in the CPS split, for example). These limitations are revisited in Section 7.2.

6.6 Discussion

Hypothesis confirmation. The central hypothesis of this thesis is that per-user variation in analog click telemetry is structured enough to support automatic labeling, statistical modeling, and parameter optimization beyond static default settings. The results across all three subsystems confirm this. The learned classifier recovers 97.0% of intended left-channel click events — a $2.5\times$ improvement over a rule-based heuristic that cannot detect firmware-unregistered presses. The optimizer reduces make latency by 11.6 ms (median 11.4 ms) across every one of 238 channel-records, with a Wilcoxon signed-rank test on the per-session loss delta ($W = 0$) confirming the improvement is consistent, not an artifact of a few large gains, and that the structure in individual telemetry is not only learnable but practically exploitable. The end-to-end user study (Section 6.5) provides a first, under-

powered but directionally consistent signal that this theoretical gain carries through to measured gameplay: a statistically significant CPS improvement, directionally favorable but inconclusive Tracking results, and a cautionary, non-significant accuracy trade-off on Flicking.

Why the RF outperforms the FCN despite its simpler architecture. The dilated residual FCN was expected to outperform the Random Forest by exploiting long-range temporal context (3,060-frame receptive field vs. a 33-frame sliding window). On the left channel the FCN falls 3.4 percentage points short of the RF (F1 93.6% vs. 97.0%) on the single held-out test set; more critically, its grouped cross-validation collapses to 59.7 ± 17.4 pp, with one fold reaching 36.7%. On the right channel the single-test gap is 8.2 points (83.6% vs. 91.8%) and the grouped CV mean is 76.0%. The right-channel result reveals the underlying cause: the FCN requires sufficient labeled examples to learn latent temporal structure from a 3-second context window, and the right-button corpus (6,645 cycles) provides insufficient coverage. On the left channel (39,088 cycles) the FCN reaches a near-identical IoU to the RF (91.5% vs. 91.6%), suggesting the two models converge to comparable boundary estimates once data is plentiful; however, the grouped cross-validation collapse shows this apparent parity does not reflect true cross-participant generalisation.

The heuristic’s IoU anomaly. An unexpected finding is that the heuristic’s IoU on matched clicks (91.6 %) matches the RF-L IoU (91.6 %) despite the RF’s far higher deployment F1. The heuristic anchors its duration estimate on the OS digital event, which provides a reliable proxy for the full biomechanical press-to-release window; when the OS event is present and the heuristic correctly matches a ground-truth click, its Make and Break boundaries track the label closely because both start from the same digital anchor. The models estimate boundaries from the analog depth signal alone, so their per-matched-click IoU gains no ground on the heuristic even though their overall detection rate is far superior. This points to a natural hybrid strategy: use the learned classifier to detect clicks the OS cannot register, and refine the duration estimate using the OS event when it is available.

Failure mode analysis. Fold 4 consistently produces the lowest deployment F1 across the tree-based models (RF-L: 91.0 %, GBT-R: 83.9 %). Each grouped cross-validation fold holds out a specific participant group, and the consistent ranking suggests the participants held out in fold 4 share an atypical click profile — likely shallow resting depth or micro-tremor-rich idle signals that cause frequent false positives or missed detections. This is a known challenge in per-user generalization: a model trained on other participants may not have seen enough examples of extreme profiles to learn reliable background rejection for that subpopulation. Active learning over the cross-validation folds — prioritizing annotation of participants whose profiles diverge most from the training distribution — is a natural direction for reducing this variance.

Heuristic per-participant heterogeneity. The geometric click-intent heuristic (Section 4.2) exhibits extreme individual variance, with per-participant F1 ranging from 0.1 % to 75.0 % on the left channel (CV=0.79) and 0.0 % to 43.5 % on the right (CV=0.82). This spread does not track cleanly with any single self-reported attribute (age, gaming genre, weekly hours, or experience level) we collected, which is itself informative: it suggests the heuristic’s reliability is governed by low-level, unmeasured biome-

chanical factors — grip pressure, resting finger depth, micro-tremor magnitude — that a fixed-shape geometric rule (peak prominence and derivative thresholds tuned on aggregate statistics) cannot adapt to on a per-user basis. A purely geometric detector is, by construction, a single global rule applied uniformly across the population; when the underlying signal shape genuinely differs across individuals, no single set of thresholds can fit all of them well, which is precisely the failure mode a learned, per-user-adaptive classifier is designed to correct.

Practical implications of the optimizer result.

The static mid-travel default (AP 5/10) assumes the user can press comfortably to 50 % of travel before actuation, incurring a mean make latency of 27.0 ms relative to the user’s biomechanical intent. The optimized AP typically moves the threshold to shallower travel, reducing make latency to 15.4 ms and enabling the firmware to register the user’s intent almost immediately after the biomechanical onset. An 11.6 ms reduction is on the order of a perceptual just-noticeable difference for individual input events, making this a plausibly perceptible responsiveness gain. Setting the detection-cost parameter to $c_{\text{det}} = 40$ ms also eliminates degenerate maximum-RT-hysteresis selections, so break latency is penalized consistently across all 132 sessions without a measurable cost to TP rate.

Architecture and training speed. For deployment in a post-session recommendation workflow, training time is a practical constraint. The RF right-channel model trains in 141 s on CPU, and the left-channel model in 23 minutes. The GBT and RF reach near-identical grouped cross-validation generalisation (mean F1 $\approx$ 93.3 % on the L channel), making the RF a strong production choice given its markedly lower training latency than the FCN. The FCN’s 44-minute training time is already measured with RTX 5080 CUDA acceleration, so no further order-of-magnitude speedup is available from hardware alone; it remains acceptable for occasional offline re-calibration but precludes on-demand per-session retraining, reinforcing the RF as the production default until the right-channel corpus is

expanded enough to justify the FCN’s architectural complexity on its own merits.

From theoretical to measured gains. The user study (Section 6.5) provides the first evidence that the optimizer’s simulator-measured latency reduction translates into observable gameplay change, at least for CPS: the calibrated group’s +10.7% click-rate gain against a placebo regression ($p = 0.035$, $d = 0.92$) is consistent with the make-latency improvement reported in Section 6.4, since a lower actuation point directly shortens the interval between successive presses in a rapid-click task. The Tracking results trend in the same direction without reaching significance, plausibly because tracking performance depends more on mouse-movement precision than on click latency, diluting the effect of a click-timing-only intervention. The Flicking result is the one point of tension with the theoretical model: a lower actuation point that improves make latency could simultaneously reduce the physical travel margin available before an unintended actuation, which would manifest as the observed (non-significant) accuracy decrease and elevated misclick count (3/13 vs. 0/13) in a fast, high-force flicking motion. This suggests the optimizer’s objective function, which currently scores actuation timing in isolation, may need to account for mouse-movement dynamics to avoid trading click-timing gains for accuracy losses in flick-dominant contexts — a concrete direction discussed further in Section 7.3.

The role of context probes. The dominance of context probes over engineered kinematic features (Section 6.3.3) indicates that the production RF is effectively operating as a probe-based trajectory pattern matcher over a 100 ms raw-depth window, with kinematic channels providing a modest regularisation signal on top. This reframes the RF-versus-FCN comparison in Section 6.2: both architectures ultimately draw most of their signal from local raw-depth context rather than from explicitly derived features, so the FCN’s much larger receptive field is not yet paying for itself at current dataset sizes.

7. Conclusion

7.1 Summary

This thesis set out to answer whether per-user variation in analog mouse-click telemetry is structured enough to support automatic labeling, statistical modeling, and firmware parameter optimization beyond static factory defaults. The answer is yes on all three fronts, confirmed by a complete pipeline evaluated on a corpus of 132 annotated recordings from 39 participants collected at two public gaming events.

Labeling pipeline. An algorithmic bootstrapping heuristic combined with iterative model-assisted annotation produced 39,088 verified left-button and 6,645 right-button click cycles at 1 kHz. The three-phase workflow (heuristic bootstrap → model-assisted labeling → human review) substantially reduced annotation burden relative to labeling from scratch while maintaining biomechanical fidelity: the heuristic alone achieves a pooled deployment F1 of only 19.9% across the full corpus, showing that human review is essential for the firmware-unregistered and noise-confounded cases that are most informative for model training.

Click-intent classification. The Random Forest achieves a deployment F1 of 97.0% on the left channel (grouped cross-validation mean 93.2 ± 1.4 pp), with make- and break-boundary bias under 1.5 ms — well within the 8–12 ms perceptual threshold. Random Forest and Gradient Boosted Trees achieve near-identical grouped cross-validation generalization, and the RF trains substantially faster on the left channel (23 minutes versus the FCN’s 44 minutes), confirming it as the default production architecture. The dilated FCN, despite a 3,060-frame receptive field, provides no measurable gain on the left channel in the single-test comparison and collapses under grouped cross-validation on that same channel (F1 59.7 ± 17.4 pp), revealing that large-context models require proportionally large annotated corpora.

Firmware threshold optimizer. The exhaustive FSM simulator reduced optimizer loss by 49% (Wilcoxon $W = 0$, $n = 238$, $p \ll 0.001$), increased

the TP rate from 90.9 % to 97.5 %, and reduced mean make latency from 27.0 ms to 15.4 ms — an 11.6 ms improvement that spans roughly 1 perceptual just-noticeable difference for input timing, and represents a directly perceptible responsiveness gain available to every user without any manual configuration.

End-to-end user study. A double-blind study with 26 participants (13 calibrated, 13 placebo) tested whether these theoretical gains are perceptible in gameplay. The calibrated group showed a statistically significant, large improvement in clicks-per-second (+10.7 % vs. −2.5 % for placebo, $p = 0.035$, $d = 0.92$), directionally favorable but inconclusive results on a tracking task, and a non-significant accuracy trade-off on a fast-flicking task accompanied by a higher misclick count. The study confirms the optimizer behaves well end-to-end — producing a healthy, non-degenerate spread of per-user recommendations — while also surfacing a concrete risk (click-timing gains trading off against flick accuracy) that the offline evaluation alone could not have revealed.

7.2 Limitations

Heuristic unreliability across participants. The heuristic bootstrap’s deployment F1 varies from 0.1 % to 75.0 % across individual participants (CV 0.79 on the left channel), and this spread does not track cleanly with any collected demographic or self-reported gaming attribute. The heuristic applies one fixed geometric rule (peak prominence and derivative thresholds tuned on aggregate statistics) uniformly across the population; when individual grip pressure, resting depth, or micro-tremor magnitude differs enough from that aggregate profile, the rule fails outright rather than degrading gracefully. This is precisely the failure mode motivating a learned, per-user-adaptive classifier over a fixed geometric one.

Right-channel data imbalance. The right-button corpus is approximately six times smaller than the left (6,645 vs. 39,088 cycles), causing the FCN and the heuristic to degrade significantly on that channel. A balanced collection protocol — or a ded-

icated right-click minigame that naturally elicits more right-button activity — is needed for production right-channel deployment.

Single hardware platform. All data was collected on a single HITS-enabled PRO X2 Superstrike mouse. Generalization to other analog input devices with different sensor resolutions, quantization levels, or firmware architectures has not been tested. The feature-engineering pipeline (120-level depth, derivative channels) is designed to be device-agnostic, but empirical transfer must be verified.

Annotation ambiguity at onset boundaries. The ± 4 ms onset tolerance absorbs natural annotator variability, but for borderline clicks at the edge of the biomechanical window the correct label boundary is genuinely ambiguous. The asymmetric soft-label targets implemented for the GBT model (Section 4.4) could partially address this, but were not used for the reported results, and a formal inter-annotator agreement study was not conducted; the reported metrics therefore include an irreducible component of label noise.

FCN hyperparameter underexploration. The dilated residual FCN was trained for only 16–17 epochs with a fixed hyperparameter set due to the cost of training (~44 minutes per run, already with RTX 5080 CUDA acceleration). No systematic search over architecture depth, dropout rate, learning rate schedule, or number of dilation stages was performed. The observed grouped cross-validation collapse (59.7 ± 17.4 pp) may therefore reflect a poorly tuned model rather than a fundamental architectural limitation. An Optuna sweep over the FCN’s hyperparameter space, combined with a larger right-button corpus, is the most likely path to closing the performance gap with the Random Forest; since GPU acceleration is already in use, further speedup would have to come from a smaller search space or distributed sweeps rather than additional hardware.

Absence of session-level temporal context. All three architectures operate on a local window of at most 100 ms of context (24-frame window plus 10 context probes). The Random Forest has no access

to session-level dynamics: the current click's depth profile is evaluated in isolation from the cumulative fatigue drift, the inter-click tempo, or the running statistics of the session so far. Biomechanical research suggests that finger press depth and velocity evolve systematically over extended play sessions, and that click intent signatures cluster differently at the start versus end of a 40-minute session; a classifier blind to this drift cannot distinguish a genuine late-session change in intent from noise. Section 7.3 outlines a concrete feature-level extension addressing this gap.

Binary click-matching criterion. The dual-radius matching criterion used for deployment F1 (Section 6.1) classifies each (predicted, ground-truth) pair as a binary hit or miss based on whether *both* onsets fall within a fixed 8 ms radius of their respective ground-truth boundary centers. This treats a prediction whose make-onset misses the radius by 1 ms the same as one that misses by 50 ms, and discards any information about near-misses that fail only one of the two radius checks. A graded similarity metric — such as a distance-weighted match score or a duration-error penalty — would provide a more informative picture of where boundary errors concentrate and would allow distinguishing a nearly correct prediction from a completely wrong one.

Underpowered user study. The end-to-end user study (Section 6.5) used 13 participants per group, well below the ~ 29 per group ($n = 58$ total) that a Lehr's-formula power calculation at the observed effect sizes ($d \approx 0.75$) indicates is needed for adequately powered ($1 - \beta = 0.80$) comparisons. Observed power on the non-CPS metrics ranged from 0.17 to 0.62. Between-subject assignment left uncontrolled variance in baseline skill level, no warmup round preceded the timed tasks (leaving results susceptible to a learning-effect confound between the calibration and test rounds), and gamer/non-gamer counts were not balanced in advance across groups. The Tracking and Flicking results should therefore be read as hints rather than conclusive findings, and the Flicking accuracy/misclick trade-off in particular warrants repli-

cation before it is treated as a genuine effect of lowering the actuation point.

7.3 Future Work

On-device inference. The current pipeline is entirely offline. Quantizing the Random Forest to INT8 and deploying it on the peripheral MCU would enable real-time per-frame classification at 1 kHz without a host connection, matching the latency requirements discussed in the Background chapter and opening the path to adaptive threshold adjustment during play.

Adaptive recalibration. Neuromotor fatigue shifts the analog depth signal over the course of a gaming session — specifically, the resting drift channel rises as the flexor muscles tire. A lightweight online loop that monitors the running mean of the resting-drift channel and nudges the actuation threshold accordingly could maintain the optimized configuration as the user's biomechanical state evolves.

Cross-device and cross-game generalization. Replicating the data collection on Hall-effect keyboards and alternative mouse switches would test whether the trained feature representations transfer, and whether a single universal model is feasible or per-device fine-tuning is required. The game-genre effect (FPS vs. MOBA click patterns, as observed in the PolyLAN dataset) is also worth modelling explicitly to produce genre-aware recommendations.

Session-aware global feature extraction. The most impactful architectural extension for the Random Forest would be to append session-level summary features to each per-frame feature vector: cumulative click count, running mean and variance of the resting-drift channel, recent inter-click interval (a proxy for play tempo and arousal), and a fatigue indicator derived from the slow drift of the resting depth baseline over the session. These features would allow the model to adapt its decision boundary as both the hardware signal and the user's neuromuscular state drift across a 40-minute session, addressing the absence of long-range temporal context that the probe ablation identified as

the primary bottleneck.

Two-stage cascade with boundary refinement.

The current pipeline treats detection and boundary placement as a single classification step. A two-stage architecture would separate these concerns: Stage 1 uses the existing RF (fast, reliable at detecting click events) to produce candidate click intervals with loose boundaries; Stage 2 applies a lightweight boundary-refinement model — for example, a shallow recurrent network or a small attention block — that takes a narrow context window around each candidate boundary and emits a sub-frame corrected onset time. The Stage 2 model could be trained on residuals from the Stage 1 predictions, focusing its capacity entirely on the hard disambiguation cases near onset boundaries.

FCN with systematic hyperparameter search.

With a larger corpus (achievable by extending collection to additional hardware platforms or broader user populations), a full Optuna sweep over the FCN’s architecture and training hyperparameters, run on the existing RTX 5080, would determine whether the left-channel performance gap is closeable. The probe ablation finding — that the RF’s 100 ms probe window captures most of the available local context — suggests that the FCN’s 3 s receptive field advantage is currently nullified by data sparsity rather than by an architectural mismatch, making data expansion the higher-leverage investment.

Adequately powered, within-subject user study.

The user study reported in Section 6.5 establishes a significant CPS effect but was underpowered to resolve the Tracking and Flicking results. A follow-up study should (1) recruit approximately 58 participants (29 per group, per the Lehr’s-formula estimate at the observed effect sizes), (2) have each participant play every task twice, once under the placebo condition and once under the calibrated condition, to reduce cross-participant skill-level variance rather than relying purely on between-subject comparison, (3) introduce a warmup round before each timed task to mitigate the learning-effect confound between the calibration and test rounds, and (4) balance gamer and non-gamer

counts across groups in advance, or explicitly match participants on self-reported skill level, insofar as recruitment constraints allow.

Movement-aware optimization objective.

The Flicking result (Section 6.5) suggests that optimizing actuation timing in isolation may trade away accuracy in movement-heavy click contexts: a lower actuation point shortens make latency but also shortens the travel margin before an unintended actuation during a fast flick. Incorporating mouse-movement telemetry (velocity or acceleration at the moment of click) into the optimizer’s objective function could let it distinguish stationary clicks, where a low actuation point is unambiguously beneficial, from in-motion clicks, where a more conservative threshold may better preserve accuracy.